



机器学习 (Machine Learning)

第8讲 梯度提升树与集成学习

中山大学附属第一医院

讲者：周倩（2013届全日制硕士）

导师：张晋昕 教授

2025年11月4日 · 广州

目录

CONTENTS

- 1 梯度提升树与集成学习概述
- 2 梯度提升树算法原理—GBDT
- 3 梯度提升高效实现—XGBoost, LightGBM, CatBoost
- 4 集成学习基础—Bagging, Boosting, Stacking
- 5 案例分析

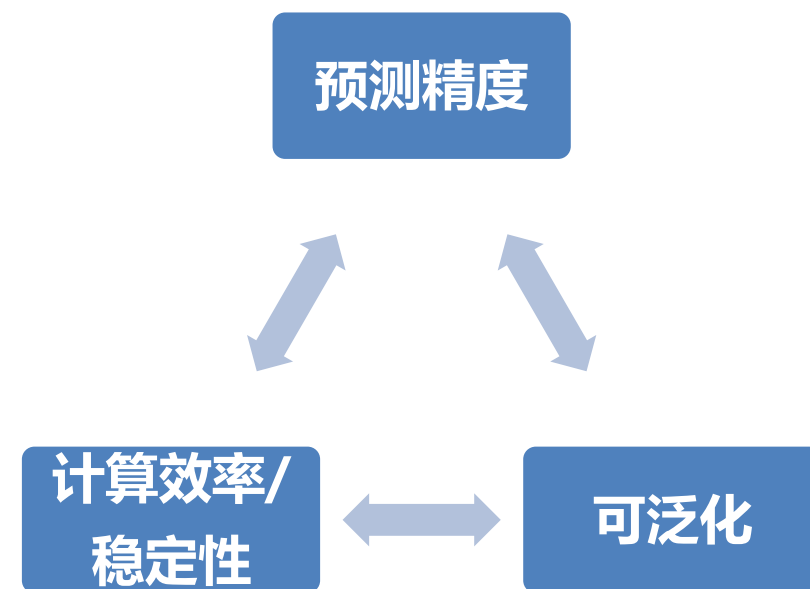
1

梯度提升树与集成学习概述

1.1 从单棵树到多棵树的集成

临床预测模型的目标-高精度、高稳健性、可泛化

- **熟悉的工具：** Logistic回归、Cox比例风险模型、线性回归
 - 可解释性强，理论基础牢固，线性模型的基石，但捕捉复杂模式的能力有限
- **已学过的机器学习模型：**
 - 决策树（直观、可解释性高、不稳定）
 - 支持向量机（核函数敏感、难扩展、可解释性存在挑战）

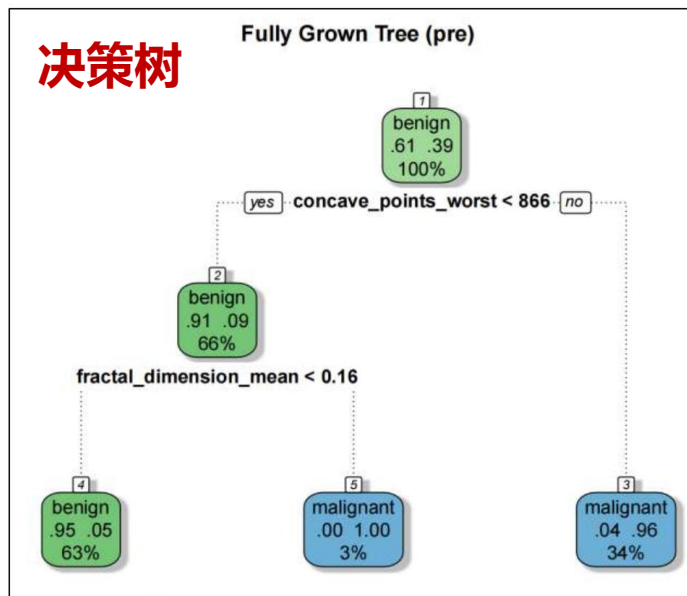


没有一个“万能”的单一模型

• ...

1.1 从单棵树到多棵树的集成

决策树



决策树根据分类依据的目标函数不同

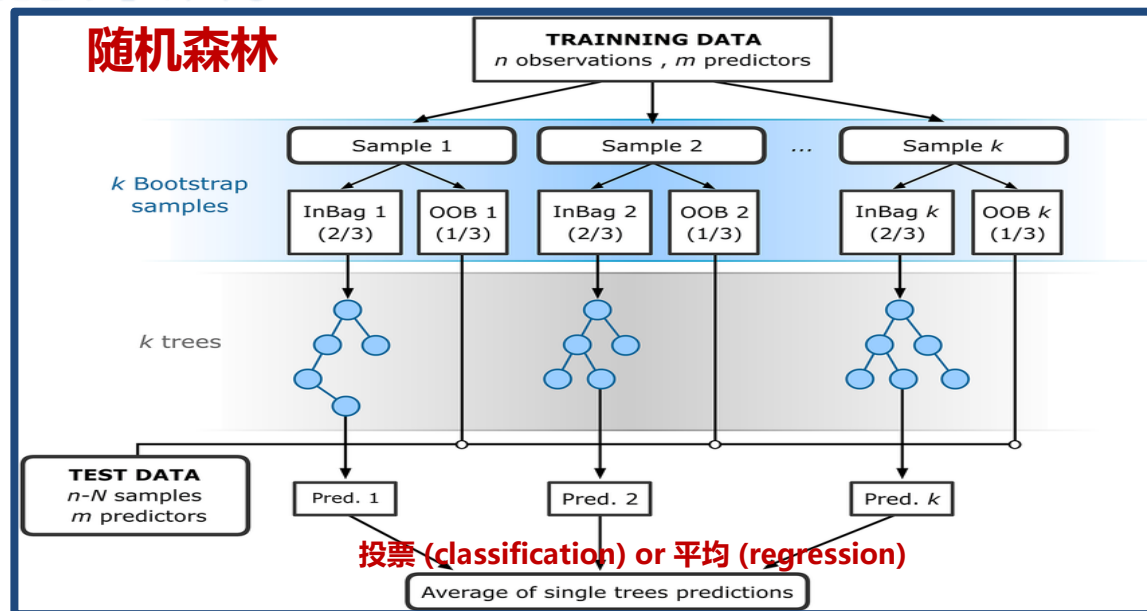
主要包含三种算法:

ID3 (Iterative Dichotomiser)、

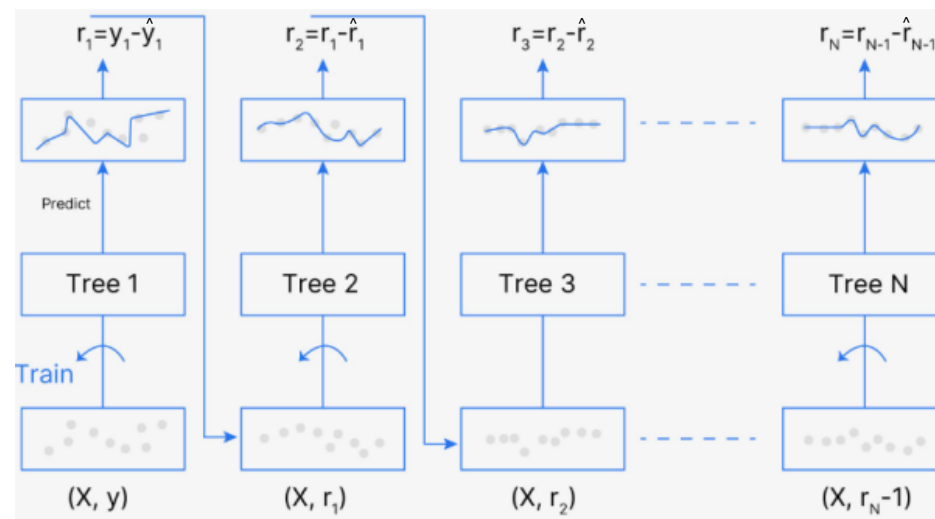
C4.5

CART (Classification And Regression Tree)

随机森林

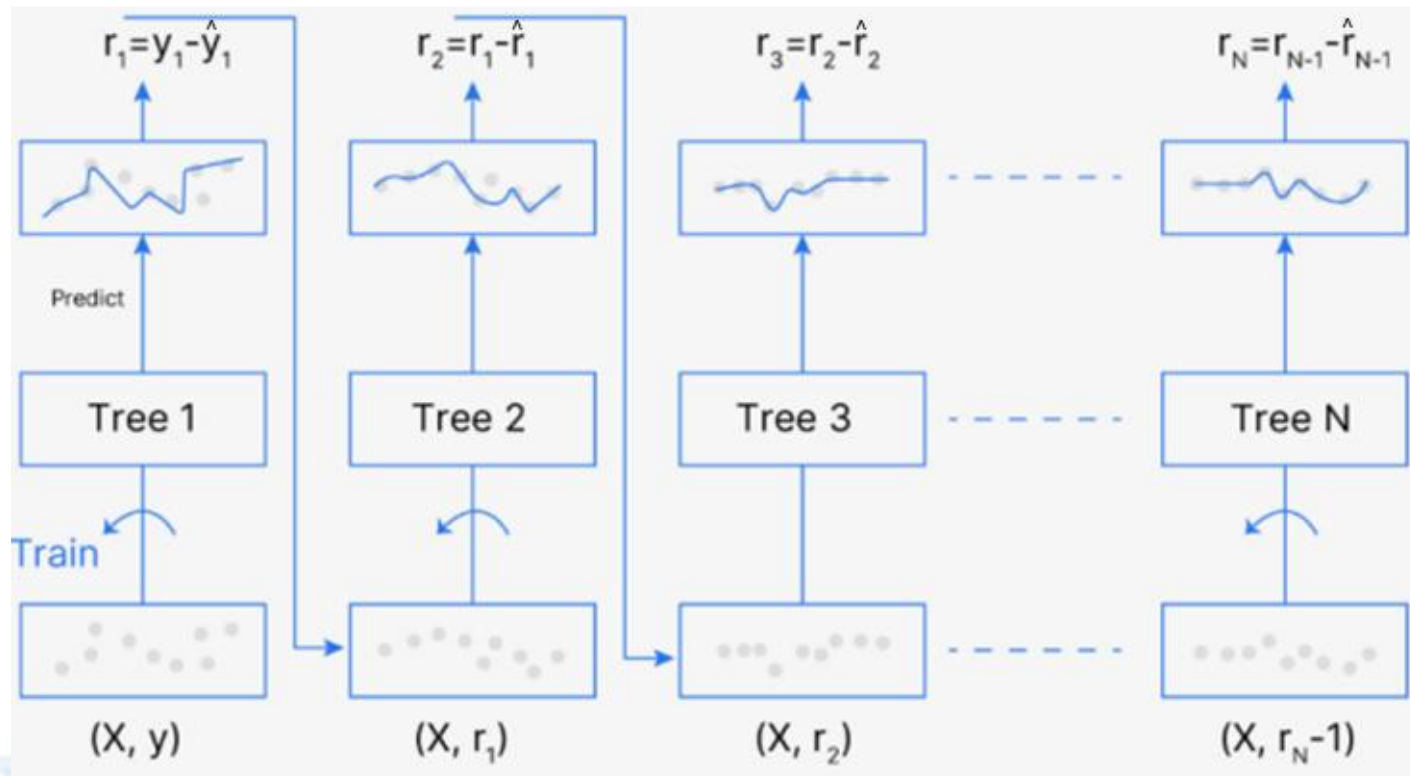


梯度提升树及其高效实现



1.2 梯度提升树简介

- **梯度提升树**（全称：Gradient Boosting Decision Tree，简称：**GBDT**）
- 通过迭代地训练**决策树**，并将前一棵树的**残差**作为下一棵树的训练目标，逐步减少预测误差





1.2 梯度提升树简介

- **梯度提升树**（全称：Gradient Boosting Decision Tree，简称：**GBDT**）
- 通过迭代地训练**决策树**，并将前一棵树的**残差**作为下一棵树的训练目标，逐步减少预测误差
- **基本思想**：沿着**损失函数的负梯度方向**，逐步构建新的决策树来修正前面模型的不足
- 用于**回归**和**分类**问题
 - 对于连续目标值回归问题，使用梯度提升回归器（**Gradient Boosting Regressor**）
 - 分类任务，采用梯度提升分类器（**Gradient Boosting Classifier**）



1.2 梯度提升树的高性能实现

梯度提升算法，在实践中通常与决策树结合，组成GBDT。针对决策树等的特点进行各种优化，得到效率更高的模型

- **XGBoost: GBDT的一个高效工程实现**
 - 二阶梯度 + 正则项 + 并行优化
- **LightGBM: 专注于速度与效率的GBDT实现**
 - 基于直方图 + Leaf-wise生长
- **CatBoost : 专注于类别特征处理的GBDT实现**
 - 类别特征处理 + 目标防泄露

1.3 梯度提升算法在医学中的应用

ORIGINAL RESEARCH ARTICLE

Machine Learning to Predict the Likelihood of Acute Myocardial Infarction

使用机器学习预测急性心肌梗死的可能性

- 使用梯度提升 (Gradient Boosting) 方法, 迭代训练一系列顺序的决策树
- 训练集N=3013, 测试集N=7998
- **输入特征:** 年龄、性别、首次及第二次肌钙蛋白I浓度、肌钙蛋白变化率
- **预测目标:** 是否患有1型心肌梗死 (二分类)
- **模型输出:** MI³(myocardial-ischemic-injury-index)值, 0-100之间, 反映个体患1型心肌梗死的可能性

- **模型表达式:**
$$\log \frac{p_1}{1 - p_1} = F_M(X) = F_0(X) + \sum_{i=1}^M a_i T_i(X, \beta_i)$$



1.3 梯度提升算法在医学中的应用



Original Investigation | Oncology

Machine Learning Approaches to Predict 6-Month Mortality Among Patients With Cancer

机器学习方法预测癌症患者6个月的死亡率

- 三种模型：logistic回归、随机森林、梯度提升方法（决策树）
- 训练+验证集：N=26525 (7:3)
- 输入特征：559个结构化EHR特征
- 预测目标：180天内是否死亡（二分类）
- 模型输出：180天死亡风险的预测概率
- 最终模型：GBDT泛化能力更好过拟合更轻、校准问题可能更容易处理、已有初步临床接受度证据

变量重要性Top20

Gradient Boosting
Albumin: last
Solid tumor: recent count
Metastatic cancer: total count
Metastatic cancer: recent count
Alkaline phosphatase: last
% lymphocytes: last
% neutrophils: last
Albumin: minimum
Alkaline phosphatase: maximum
% lymphocytes: minimum
RDW: last
Patient age
MCV: standard deviation
% neutrophils: maximum
Solid tumor: total count
Albumin: maximum
% lymphocytes manual: minimum
Sodium: minimum
ALT: count
Alkaline phosphatase: standard deviation

Table 2. Performance Metrics of Machine Learning Models^a

Algorithm	Positive Predictive Value ^b	AUC ^b	Accuracy	Specificity
Random forest	0.513 ^c	0.88 ^c	0.96 ^c	0.99 ^c
Gradient boosting classifier	0.494	0.87	0.96 ^c	0.99 ^c
Logistic regression	0.447	0.86	0.95	0.99 ^c



1.3 梯度提升算法在医学中的应用

预后预测模型

Home | JAMA Oncology | Vol. 6, No. 11

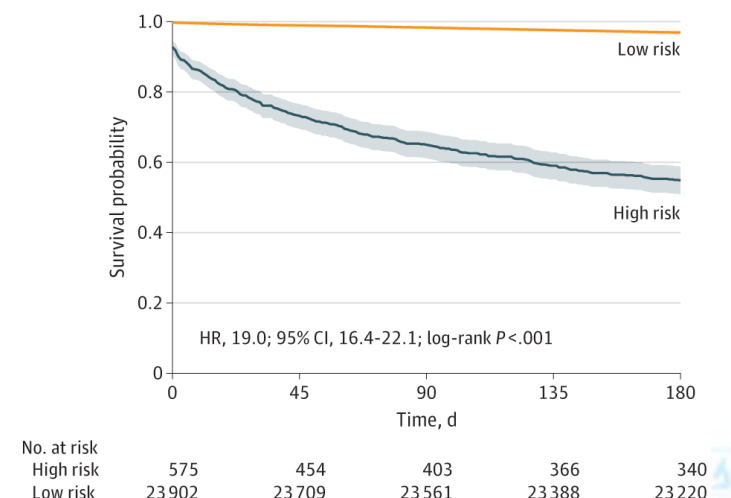
Original Investigation

FREE

Validation of a Machine Learning Algorithm to Predict 180-Day Mortality for Outpatients With Cancer

验证一种机器学习算法以预测癌症门诊患者的180天死亡率

- 梯度提升方法（决策树）
- 前瞻性外部验证，N=24582，模型整合到EHR中实时运行
- 输入特征：559个结构化EHR特征
- 预测目标：180天内是否死亡（二分类）
- 模型输出：180天死亡风险的预测概率



模型预测高风险 vs 低风险患者的
180天死亡率KM曲线



1.3 梯度提升算法在医学中的应用

ORIGINAL ARTICLE

Using Machine Learning (XGBoost) to Predict Outcomes After
Infrainguinal Bypass for Peripheral Artery Disease

使用机器学习（XGBoost）预测外周动脉疾病患者腹股沟下旁路术后结局

- 6种模型：XGBoost、随机森林、朴素贝叶斯、支持向量机、多层感知器神经网络、logistic回归
- 训练+验证集：N=59784 (7:3)
- 输入特征：术前（68个）：人口学、合并症、功能状态、影像学、用药等。术中（13个）：麻醉方式、手术时间、失血量等。术后（16个）：并发症、住院天数、出院用药等
- 预测目标：1年不良肢体事件或死亡（二分类）
- 模型输出：终点事件的预测概率

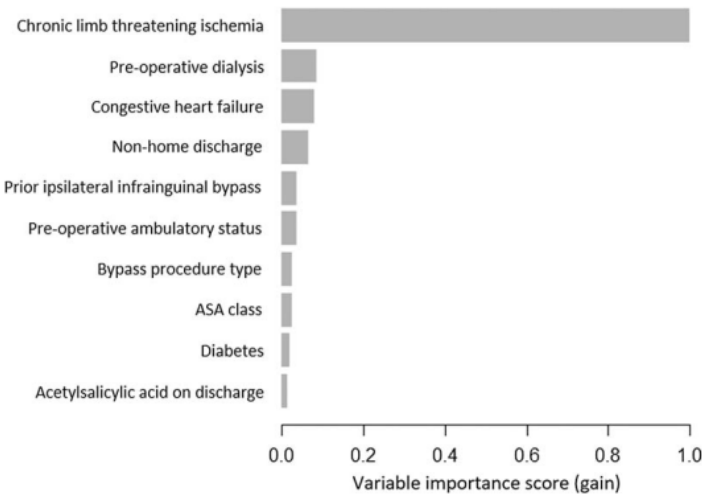


FIGURE 3. Variable importance scores (gain) for the top 10 predictors of 1-year major adverse limb event or death following infrainguinal bypass in the Extreme Gradient Boosting (XGBoost) model at the postoperative stage.



1.3 梯度提升算法在医学中的应用

诊断预测模型

Article

<https://doi.org/10.1038/s41467-024-48956-0>

Development of a novel non-invasive biomarker panel for hepatic fibrosis in MASLD

开发一种用于代谢功能障碍相关脂肪性肝病 MASLD肝纤维化的新型非侵入性生物标志物模型

- **LightGBM**
- 训练+验证集N=90+38，外部验证集N=156
- **输入特征：** 11个血清生物标志物的浓度（三大类IGFBP7、SSc5D、ema4D）
- **预测目标：** 肝纤维化分期（F0/F1、F2、F3/F4）（三分类）
- **模型输出：** 纤维化预测类别

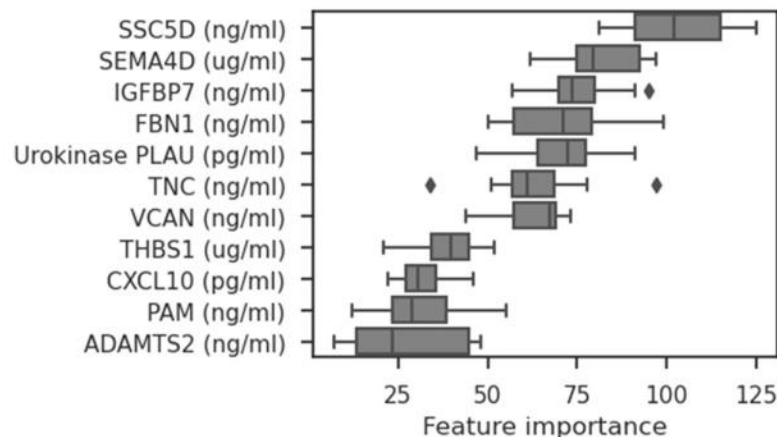


Fig. 5 | Biomarker selection. Box and Whisker plot showing feature importance list of the machine learning model, showing the contribution of each of the 11 serum biomarkers to the classification into fibrosis groups in 128 patients of the testing cohort. Data are median (vertical line), interquartile range (boxes) and 10–90% percentile (error bars). The diamonds indicate outliers.



1.3 梯度提升算法在医学中的应用

诊断预测模型

<https://doi.org/10.1038/s41746-025-01880-z>

Clinical decision support for vestibular diagnosis: large-scale machine learning with lived experience coaching

前庭诊断的临床决策支持：基于大规模机器学习与生活经验指导

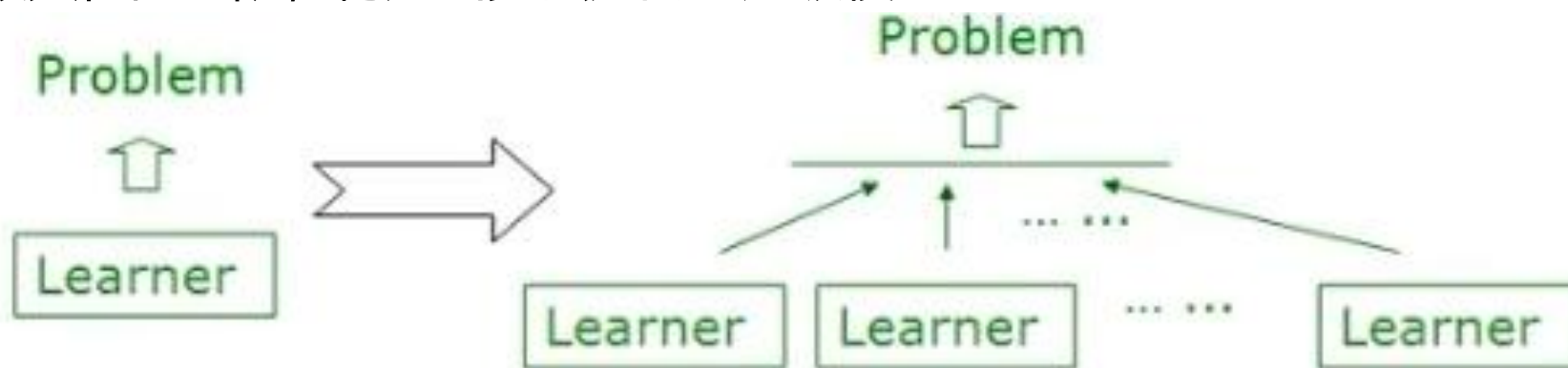
- **CatBoost（最终选择），并与Random Forest、XGBoost比较**
- 训练+验证集N=3349(8:2)
- **输入特征：** 50个临床特征
- **预测目标：** 六种前庭障碍的诊断（多分类）
- **模型输出：** 最可能的前两个诊断及其概率

Model	Validation Set	Unseen Test Set
Catboost	Total: 536	Total: 670
	Correct: 375	Correct: 408
	Part Correct: 124	Part Correct: 184
	Incorrect: 37	Incorrect: 78
	Accuracy: 0.93	Accuracy: 0.88
Random Forest	Total: 536	Total: 670
	Correct: 449	Correct: 401
	Part Correct: 76	Part Correct: 171
	Incorrect: 11	Incorrect: 98
	Accuracy: 0.98	Accuracy: 0.85
XGBoost	Total: 536	Total: 670
	Correct: 394	Correct: 406
	Part Correct: 114	Part Correct: 183
	Incorrect: 28	Incorrect: 81
	Accuracy: 0.95	Accuracy: 0.88



1.4 集成学习基础

- 随机森林：一种基于**bagging**框架的集成学习算法（并行）
- 梯度提升树：一种基于**boosting**框架的集成学习算法（顺序）
- **集成学习 (ensemble learning)**：通过构建并结合多个**个体学习器**来完成学习任务。个体学习器称为**基学习器**
 - **核心思想**：群体智慧大于个体智慧。通过系统性地组合已知的模型（如C4.5决策树），能够突破其性能上限，构建出更强大的“超级模型”



Q：如何构建多个学习器？如何系统结合这些学习器？



1.4 集成学习基础

方法类型

学习器关系

结合方式

目标

个体学习器
“好而不同”

- **并行化方法：独立、并行**

- 同质并行：Bagging、Random Forest
- 异质并行：Stacking(堆叠)
-

平均/投票
加权/学习法

↓ 方差

- **序列化方法：依赖、串行**

- 自适应提升：Adaboost
- 梯度提升：XGBoost、LightGBM、CatBoost
-

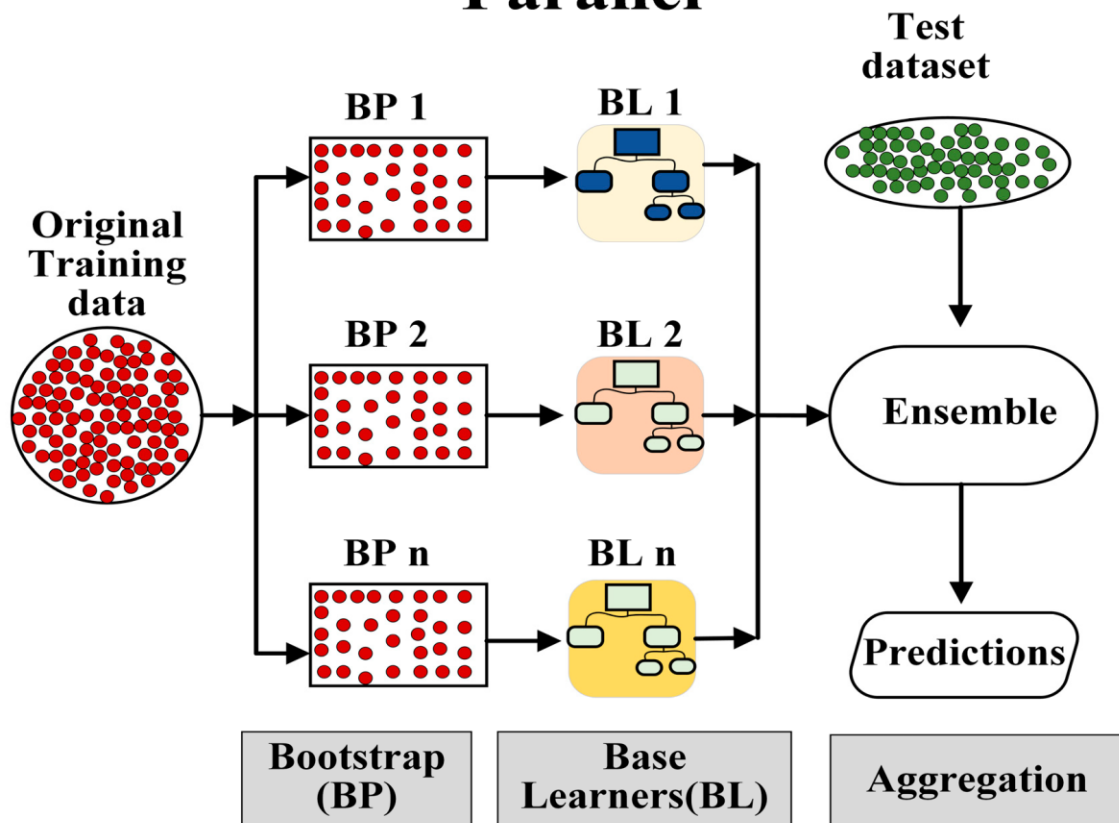
加权法

↓ 偏差

1.4 集成学习基础

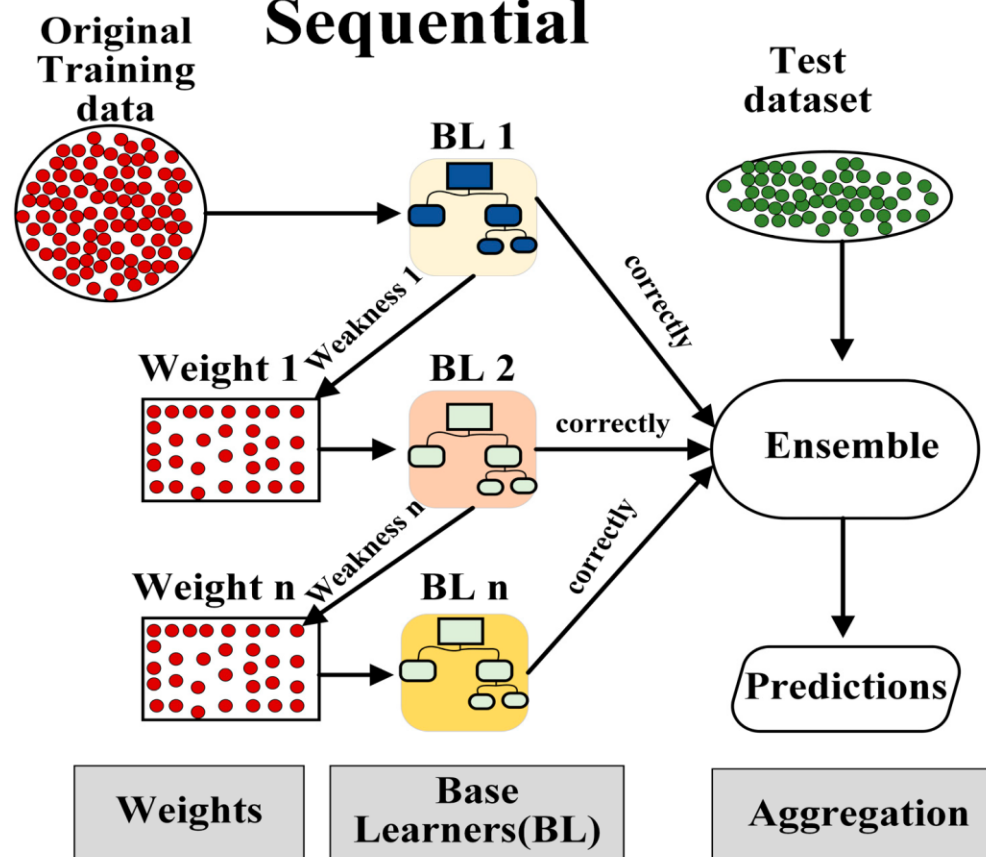
并行化方法

Parallel



序列化方法

Sequential





Next

- 梯度提升树算法原理
 - 梯度提升算法
 - GBDT
 - XGBoost、LightGBM、CatBoost
- 集成学习基础
 - Bagging vs Boosting
 - 并行化方法 vs 序列化方法
 - 数据融合策略：平均、加权以及Stacking

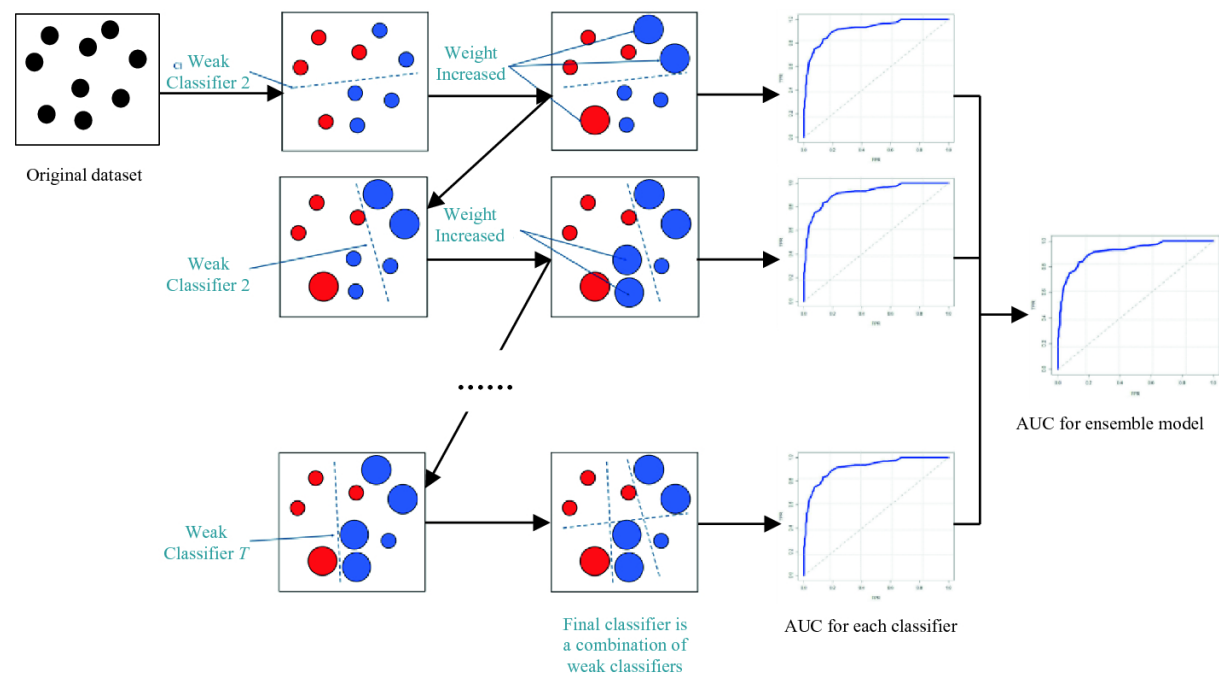


2

梯度提升树算法原理—GBDT

2.1 提升算法 (Boosting)

- 梯度提升树 (Gradient Boosting Decision Tree, GBDT) 是 Boosting (提升算法) 的典型代表
 - Boosting基本思路：利用当前模型的**偏差**来调整训练集数据的权重，使下一个模型更多关注目前偏差较大的部分，采用加法模型（基学习器的线性组合）与前向分布算法，通过构建多个**弱学习器（基学习器）**并**加权求和**最终得到**强学习器**
 - 几个关键问题：
 - 如何计算偏差？
 - 如何通过偏差计算样本权重？
 - 如何将得到的个体学习器结合？



自适应提升 (Adaboost)、

梯度提升 (Gradient Boosting) ...

2.2 自适应提升 (AdaBoost)

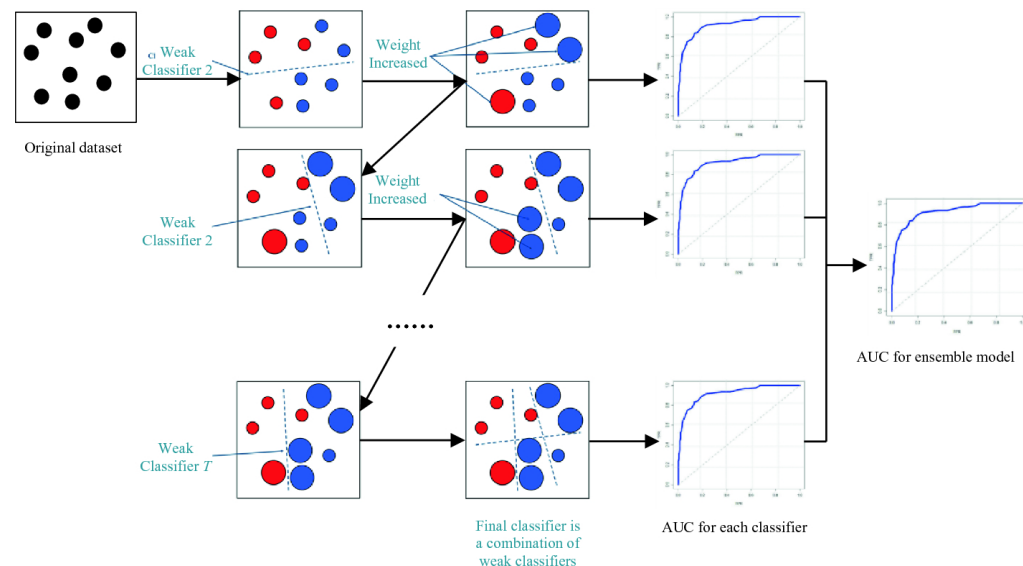
- 基本的AdaBoost算法用于**二分类问题**，以**指数损失函数**为目标
- 考虑**加法模型**表示强学习器： $F(x) = \sum_1^M a_i f_i(x) = \sum_1^M T(x; \Theta_m)$ ， M 是决策树的数量
- 基本思想：前向分步算法、增加错分样本的权重**

- 初始时所有样本具有相同的权重
- 在每轮对基学习器调用时，对样本集中每个**样本**在损失函数中的**权重**进行调整
 - 被正确分类的样本给予较小的权重
 - 错误分类的样本量权重增加
- 经过多轮得到多个基学习器 $f_i(x), i = 1, 2, \dots, M$,
 - $f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$
- 最终各基学习器的输出**加权投票**产生分类输出
 - $F(x) = \text{sgn}[\sum_1^M a_i f_i(x)]$

- 最小化损失函数可得到第 m 棵树的参数估计

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \sum_1^N L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m))$$

- 特点：**具有一定的抗过拟合能力，误差随提升过程快速下降
- 不足：**对异常值过度关注、顺序依赖强、对一般的损失函数优化困难

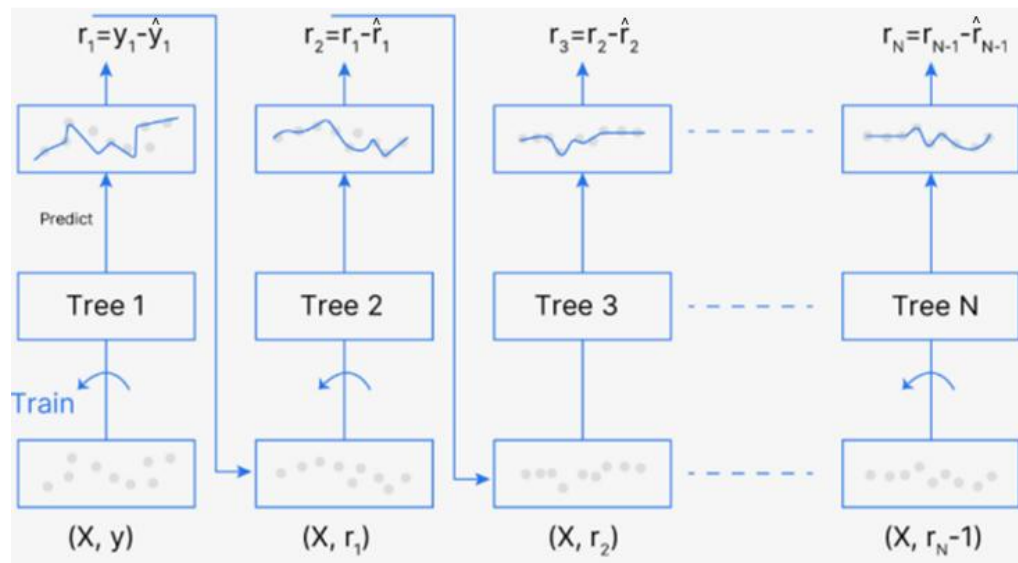


2.3 梯度提升 (Gradient Boosting)

- 仍用**加性模型**表示强学习器: $F(\mathbf{x}) = \sum_1^M a_i f_i(\mathbf{x})$, M 是弱学习器的数量
- 第 m 步时 $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + f_m(\mathbf{x})$
- 当添加第 m 个弱学习器 f_m 后, 模型在数据集上的总损失为
 - $\mathcal{L}(F_m) = E_x \left(l \left(y, \sum_1^m f_i(\mathbf{x}) \right) \right) = E_x (l(y, F_{m-1}(\mathbf{x}) + f_m(\mathbf{x})))$
- 新添加的 f_m 应使总损失尽可能小, 即最小化损失函数
 - $f_m = \arg \min_f E_x \left(l(y, \hat{y}_{m-1} + f_m(\mathbf{x})) \right)$, 其中 $\hat{y}_{m-1} = F_{m-1}(\mathbf{x})$
- 根据**梯度下降**原理: 沿损失函数梯度的反方向更新参数是让其下降最快的办法
- 根据当前模型**损失函数的负梯度**信息来训练新加入的弱分类器

2.3 梯度提升 (Gradient Boosting)

- 若为均方误差损失则 l 的形式为: $l(y, \hat{y}_{m-1} + f_m(x)) = \frac{1}{2} (y - \hat{y}_{m-1} - f_m(x))^2$
- 损失函数的一阶梯度为: $-(y - \hat{y}_{m-1})$ **残差**



GBDT通过迭代地训练**决策树**，并将前一棵树的**残差**作为下一棵树的训练目标，逐步减少预测误差
沿着**损失函数的负梯度方向**，逐步构建新的**决策树**来修正前面模型的不足，将训练好的弱分类器以累加的形式结合到现有模型中

2.3 梯度提升 (Gradient Boosting)

- 根据**梯度下降**原理：沿损失函数梯度的反方向更新参数是让其下降最快的办法
- 若将 $F(x)$ 看作损失函数 $\mathcal{L}(F)$ 的参数，则新添加的 f_m 就应当沿着 $\mathcal{L}$ 在 F_{m-1} 处的梯度方向，分别求样本 x 上预测值 $F(x)$ 的梯度： $\nabla_{F(x)} \mathcal{L}(\hat{y}_{m-1}) = \nabla_{F(x)} l(y, \hat{y}_{m-1})$
- 在**函数空间**中，梯度下降的更新规则为： $F_m(x) = F_{m-1}(x) - \eta_m \nabla_{F(x)} \mathcal{L}(\hat{y}_{m-1})$ ， η_m 为学习率（步长）
- 定义**伪残差**： $r_m(x) = -\nabla_{F(x)} \mathcal{L}(\hat{y}_{m-1})$ ，表示 $F_{m-1}(x)$ 在样本 x 上的预测误差方向
- 训练一个弱学习器拟合伪残差： $f_m(x) = \arg \min_f \sum (r_m - f(x))^2$
- $f_m(x) \approx r_m(x) = -\nabla_{F(x)} \mathcal{L}(\hat{y}_{m-1})$ 故可通过求损失函数在第 $m-1$ 步时的**梯度**来得到 f_m
- 更新整体模型： $F_m(x) = F_{m-1}(x) + \eta_m \cdot f_m(x)$ ， $0 < \eta \leq 1$

由于采用了梯度作为优化目标，因此被称为梯度提升算法

2.3 梯度提升 (Gradient Boosting)

或将损失函数 $\mathcal{L}(F_m) = \mathcal{L}(\hat{y}_{m-1} + f_m(\mathbf{x}))$ 在 $\hat{y}_{m-1}$ 处进行泰勒展开:

- $\mathcal{L}(\hat{y}_{m-1} + f_m(\mathbf{x})) = \mathcal{L}(\hat{y}_{m-1}) + g_{m-1}(\mathbf{x}) \cdot f_m(\mathbf{x}) + \frac{1}{2!} h_{m-1}(\mathbf{x}) \cdot f_m^2(\mathbf{x}) + \dots$
- 保留**一阶近似**: $\mathcal{L}(\hat{y}_{m-1} + f_m(\mathbf{x})) \approx \mathcal{L}(\hat{y}_{m-1}) + g_{m-1}(\mathbf{x}) \cdot f_m(\mathbf{x})$
- 寻找使 $\mathcal{L}(\hat{y}_{m-1} + f_m(\mathbf{x}))$ 最小的 $f_m(\mathbf{x})$, 由于 $\mathcal{L}(\hat{y}_{m-1})$ 是常数, 这等价于最小化
 - $\min_f \sum_1^N g_{m-1}(x_i) \cdot f_m(x_i)$
- 对于**每一个样本** $x_i, i = 1, 2, \dots, N$, 最优 $f_m(\mathbf{x})$ 应与 $-g_{m-1}(\mathbf{x})$ 同方向, 即 $f_m(\mathbf{x}) \propto -g_{m-1}(\mathbf{x})$
- f_m 必须来自某个**假设空间** $\mathcal{H}$ (如决策树), 因此通过以下优化问题来近似:
 - $f_m = \operatorname{argmin}_{f \in \mathcal{H}} \sum_1^N (f_m(x_i) + g_{m-1}(x_i))^2$
- 等价于让 $f_m(\mathbf{x})$ 拟合负梯度方向: $f_m(\mathbf{x}) \approx -g_{m-1}(\mathbf{x})$
- 更新模型: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta_m \cdot f_m(\mathbf{x})$, $0 < \eta \leq 1$



2.3 梯度提升 (Gradient Boosting)

- GBDT分类任务：常选择对数损失 (Log Loss) 作为损失函数
- 一阶泰勒展开： $l(y, F) = y \log(1 + e^{-F}) + (1 - y) \log(1 + e^F)$
- 梯度：
$$\frac{\partial l}{\partial F} = \sigma(F) - y = \frac{1}{1 + e^{-F}} - y$$
- 伪残差 (负梯度)：
$$r_m = y - \sigma(F_{m-1}(\mathbf{x}))$$



2.4 GBDT及前向分步算法

- **GBDT** (Gradient Boosting Decision Tree)
 - 梯度提升算法使用基模型为分类回归树 (classification and regression tree, **CART**) 的实现
 - GBRT (Gradient Boosting Regression Tree) : 梯度提升回归树, GBDT用于回归模型, 主要区别是损失函数不同 (GBDT distribution = "bernoulli" , GBRT distribution = "gaussian")
- **GBDT是通过加法模型和前向分步算法来逐步拟合负梯度**
 - **加法模型**: 允许一步步地添加组件
 - **前向分步算法**: 对模型进行分步优化
 - **拟合负梯度**: 每一棵新树的任务是拟合当前模型预测的损失函数的负梯度

2.4 GBDT及前向分步算法

• 步骤1：初始化模型

- $F_0(x) = \arg \min_c \sum_1^N (l(y_i, c))$

• 步骤2：迭代构建 M 棵决策树 ($m = 1, 2, \dots, M$)

- 计算伪残差 (负梯度) $r_{im} = -[\nabla_{F(x_i)} l(y_i, F(x_i))]_{F(x)=F_{m-1}(x)}$

- 用决策树拟合伪残差

- 第 m 步决策树 $f_m(x)$ 来拟合 $\{(x_i, r_{im})\}$, 即训练 $f_m(x)$ 来近似 r_{im}

- 为每个叶节点计算最优输出值 ($R_{jm}, j = 1, 2, \dots, J, J$ 为叶节点数)

计算使得损失最小的输出值

- $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} (l(y_i, F_{m-1}(x_i) + \gamma))$

- 更新模型为 $F_m(x) = F_{m-1}(x) + \eta \sum_j (\gamma_{jm} \cdot I(x \in R_{jm}))$

• 步骤3：输出最终模型

- $F_M(x) = F_0(x) + \eta \sum_{m=1}^M \sum_j (\gamma_{jm} \cdot I(x \in R_{jm}))$

- 平方损失: $r_{im} = y_i - F_{m-1}(x_i)$ (就是普通的残差)

- 绝对损失: $r_{im} = \text{sign}(y_i - F_{m-1}(x_i))$ (符号函数)

- Huber损失: 在残差较小时是残差, 较大时是带符号的常数

- 对数损失 (二分类): $r_{im} = y_i - \frac{1}{1+e^{-F_{m-1}(x_i)}}$ (真实标签与预测概率的差)

2.5 GBDT正则化技术

- **学习率收缩 (Shrinkage)**：通过减小每棵树的贡献来降低模型复杂度

- $F_m(x) = F_{m-1}(x) + f_m(x)$

- $F_m(x) = F_{m-1}(x) + \eta_m \cdot f_m(x)$, $0 < \eta \leq 1$, 学习率

- 常用范围：0.01 - 0.1, 学习率越小, 需要的树越多

- 交叉验证：通过网格搜索确定最优组合

- **子采样 (Subsampling)**：减少过拟合, 加速训练

- 每次迭代只使用部分样本或特征进行训练, 行子采样、列子采样

- 选择小于1的比例会增加样本拟合的偏差, 推荐0.5-0.8

- 使用了子采样的 GBDT 有时也称作随机梯度提升树 (Stochastic Gradient Boosting Tree, SGBT)

- **早停 (Early Stopping)**：模型在**验证集**错误率不再下降, 就停止训练

- **树的结构控制**, 如CART回归树进行剪枝

2.6 GBDT超参数与模型控制

- M : 树的数量 *n.trees* → 与学习率配合, 3000-10000
- η : 学习率 *shrinkage* → 小 η +多树更稳健, 0.001-0.1
- 树深度 *interaction.depth* → 控制单棵树复杂度
- 叶节点最小样本树 *n.minobsinnode* → 1-10
- 样本量采样比例 *bag.fraction* → 每次迭代中随机抽取的训练数据, 引入随机性, 即随机梯度提升subsample, 减小方差
- 损失函数类型 *distribution* (回归: 均方差gaussian、绝对损失laplace; 二分类: 对数损失bernoulli、铰链损失huberized、指数损失adaboost; 多分类: multinomial; 生存分析coxph...)
- 早期停止: 使用cv.folds确定确定早停点

R gbm包

```
gbm(  
  formula = formula(data),  
  distribution = "bernoulli",  
  data = list(),  
  weights,  
  var.monotone = NULL,  
  n.trees = 100,  
  interaction.depth = 1,  
  n.minobsinnode = 10,  
  shrinkage = 0.1,  
  bag.fraction = 0.5,  
  train.fraction = 1,  
  cv.folds = 0,  
  keep.data = TRUE,  
  verbose = FALSE,  
  class.stratify.cv = NULL,  
  n.cores = NULL  
)
```


3

梯度提升高效实现—XGBoost, LightGBM, CatBoost

3.1 XGBoost: eXtreme Gradient Boosting

- **极限梯度提升：基于GBDT实现的集成算法，提升速度和效率**
 - 适用于任何损失函数，即可使用弱学习器改善任何分类、回归、排序问题
 - 在损失函数中添加**与决策树复杂度有关的正则化约束**，防止单个决策树发生过拟合现象
 - 设第 m 步拟合决策树 f_m ，其复杂度为 $\Omega(f_m)$ ，则模型的损失函数为
 - $\mathcal{L}(F_m) = \sum_x l(y, F_{m-1}(\mathbf{x}) + f_m(\mathbf{x})) + \Omega(f_m)$
 - 泰勒展开并保留到**二阶梯度**

$$\mathcal{L}(F_m) \approx \sum_x \left(l(y, \hat{y}_{m-1}) + g(\mathbf{x}) f_m(\mathbf{x}) + \frac{1}{2} h(\mathbf{x}) f_m^2(\mathbf{x}) \right) + \Omega(f_m)$$

3.1 XGBoost: eXtreme Gradient Boosting

- 极限梯度提升：基于GBDT实现的集成算法，提升速度和效率

- 复杂度导出的正则化约束可写为

- $\Omega(f_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$ 或 $\Omega(f_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 + \alpha \sum_{j=1}^J |w_j|$,

其中 T 为决策树的叶节点数目, $w \in \mathbb{R}^T$ 为每个叶节点上的预测值组成

的向量 (也叫权重), γ (gamma, 叶子数量惩罚)、 λ (lambda,

 L2正则化)、 α (alpha, L1正则化) 为正则化约束强度, 值越大代表

损失函数越关注控制模型的复杂度, γ 和 λ 最常用

- 其他优化：并行优化、缓存优化等, 可自动处理缺失值、允许特征子采样

R xgboost包中XGBoost模型训练的参数

params

the list of parameters. The complete list of parameters is available in the [online documentation](#). Below is a shorter summary:

1. General Parameters

- `booster` which booster to use, can be `gbtree` or `gblinear`. Default: `gbtree`.

2. Booster Parameters

2.1. Parameters for Tree Booster

- `eta` control the learning rate: scale the contribution of each tree by a factor of $0 < \eta < 1$ when it is added to the current approximation. Used to prevent overfitting by making the boosting process more conservative. Lower value for `eta` implies larger value for `nrounds`: low `eta` value means model more robust to overfitting but slower to compute. Default: 0.3
- `gamma` minimum loss reduction required to make a further partition on a leaf node of the tree. the larger, the more conservative the algorithm will be.
- `max_depth` maximum depth of a tree. Default: 6
- `min_child_weight` minimum sum of instance weight (hessian) needed in a child. If the tree partition step results in a leaf node with the sum of instance weight less than `min_child_weight`, then the building process will give up further partitioning. In linear regression mode, this simply corresponds to minimum number of instances needed to be in each node. The larger, the more conservative the algorithm will be. Default: 1
- `subsample` subsample ratio of the training instance. Setting it to 0.5 means that xgboost randomly collected half of the data instances to grow trees and this will prevent overfitting. It makes computation shorter (because less data to analyse). It is advised to use this parameter with `eta` and increase `nrounds`. Default: 1
- `colsample_bytree` subsample ratio of columns when constructing each tree. Default: 1
- `lambda` L2 regularization term on weights. Default: 1
- `alpha` L1 regularization term on weights. (there is no L1 reg on bias because it is not important). Default: 0
- `num_parallel_tree` Experimental parameter. number of trees to grow per round. Useful to test Random Forest through XGBoost (set `colsample_bytree` < 1 , `subsample` < 1 and `round` = 1) accordingly. Default: 1
- `monotone_constraints` A numerical vector consists of 1, 0 and -1 with its length equals to the number of features in the training data. 1 is increasing, -1 is decreasing and 0 is no constraint.
- `interaction_constraints` A list of vectors specifying feature indices of permitted interactions. Each item of the list represents one permitted interaction where specified features are allowed to interact with each other. Feature index values should start from 0 (0 references the first column). Leave argument unspecified for no interaction constraints.

2.2. Parameters for Linear Booster

- `lambda` L2 regularization term on weights. Default: 0

- `lambda_bias` L2 regularization term on bias. Default: 0
- `alpha` L1 regularization term on weights. (there is no L1 reg on bias because it is not important). Default: 0

3. Task Parameters

- `objective` specify the learning task and the corresponding learning objective, users can pass a self-defined function to it. The default objective options are below:
 - `reg:squarederror` Regression with squared loss (Default).
 - `reg:squaredlogerror`: regression with squared log loss $1/2 * (\log(pred + 1) - \log(label + 1))^2$. All inputs are required to be greater than -1. Also, see metric `rmsle` for possible issue with this objective.
 - `reg:logistic` logistic regression.
 - `reg:pseudohubererror`: regression with Pseudo Huber loss, a twice differentiable alternative to absolute loss.
 - `binary:logistic` logistic regression for binary classification. Output probability.
 - `binary:logitraw` logistic regression for binary classification, output score before logistic transformation.
 - `binary:hinge`: hinge loss for binary classification. This makes predictions of 0 or 1, rather than producing probabilities.
 - `count:poisson`: Poisson regression for count data, output mean of Poisson distribution. `max_delta_step` is set to 0.7 by default in poisson regression (used to safeguard optimization).
 - `survival:cox`: Cox regression for right censored survival time data (negative values are considered right censored). Note that predictions are returned on the hazard ratio scale (i.e., as $HR = \exp(\text{marginal_prediction})$ in the proportional hazard function $h(t) = h_0(t) * HR$).
 - `survival:aft`: Accelerated failure time model for censored survival time data. See [Survival Analysis with Accelerated Failure Time](#) for details.
 - `aft_loss_distribution`: Probability Density Function used by `survival:aft` and `aft-nloglik` metric.
 - `multi:softmax` set xgboost to do multiclass classification using the softmax objective. Class is represented by a number and should be from 0 to `num_class - 1`.
 - `multi:softprob` same as softmax, but prediction outputs a vector of `ndata * nclass` elements, which can be further reshaped to `ndata, nclass` matrix. The result contains predicted probabilities of each data point belonging to each class.
 - `rank:pairwise` set xgboost to do ranking task by minimizing the pairwise loss.
 - `rank:ndcg`: Use LambdaMART to perform list-wise ranking where [Normalized Discounted Cumulative Gain \(NDCG\)](#) is maximized.
 - `rank:map`: Use LambdaMART to perform list-wise ranking where [Mean Average Precision \(MAP\)](#) is maximized.

- `reg:gamma`: gamma regression with log-link. Output is a mean of gamma distribution. It might be useful, e.g., for modeling insurance claims severity, or for any outcome that might be [gamma-distributed](#).
- `reg:tweedie`: Tweedie regression with log-link. It might be useful, e.g., for modeling total loss in insurance, or for any outcome that might be [Tweedie-distributed](#).

- `base_score` the initial prediction score of all instances, global bias. Default: 0.5
- `eval_metric` evaluation metrics for validation data. Users can pass a self-defined function to it. Default: metric will be assigned according to objective (rmse for regression, and error for classification, mean average precision for ranking). List is provided in detail section.

3.2 LightGBM: Light Gradient Boosting Machine

- **轻量级梯度提升机：GBDT的一种改进方法**
 - XGBoost要遍历所有特征来找最佳分裂点，数据多特征多时速度慢
 - LightGBM先把**连续型特征**分箱（binning）成**直方图**，用直方图来找分裂点，速度提升很多（**参数max_bin、min_data_in_bin**）
 - 一种快速的、分布式的、高性能的梯度提升框架，基于决策树算法，可用于排序、分类及许多其他机器学习任务，目标函数包含：
 - 回归: "regression", "regression_l1", "huber"
 - 二分类: "binary"
 - 多分类: "multiclass"
 - 排序: "lambdarank"

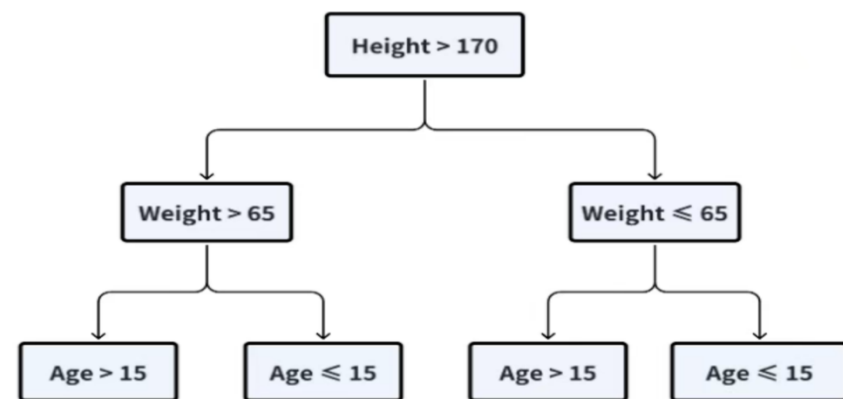
3.2 LightGBM: Light Gradient Boosting Machine

• 轻量级梯度提升机: GBDT的一种改进方法

- LightGBM可通过对参数boosting_type指定梯度提升算法的具体实现框架
 - “**gbd**t” (默认值), 标准的 Gradient Boosting Decision Tree, 按顺序构建每一棵树, 每棵树都学习之前所有树的残差
 - “**dart**”, Dropouts meet Multiple Additive Regression Trees, 在每一轮迭代中随机“丢弃”之前构建的部分树, 适用于训练集与测试集分布差异大
 - “**goss**”, Gradient-based One-Side Sampling, 保留梯度较大的样本, 对梯度较小的样本进行随机采样, 适用于大样本计算资源有限
 - “**rf**”, Random Forest, 每棵树使用 bagging 采样, 特征采样, 适用于并行计算

3.3 CatBoost: Categorical Boosting

- **类别特征梯度提升树，专注于类别特征处理的GBDT实现**
 - 可高效合理地处理**分类**特征：无序分类、有序分类特征的编码
 - 使用**无意识决策树**（Oblivious Decision Trees (ODTs)，又称**对称决策树**）作为基学习器，比XGBoost和LightGBM速度更快
 - 同一层的所有节点使用相同的特征和阈值、完全平衡对称
 - 平衡的、不易过拟合、允许在测试过程中大幅加快速度
 - 使用**有序提升**（Ordered Boosting）作为其底层集成学习算法，因此能在训练期间隐式地处理**目标泄露**问题和**预测偏移**问题，提高了算法的准确性和泛化能力



3.3 CatBoost: Categorical Boosting

• 无序分类、有序分类特征的编码

- 无监督-标签编码 (Label Encoding) , 如123, 适合有序分类变量
- 无监督-独热编码 (one-hot Encoding) , 适合无序分类变量
- 有监督: 使用标签对分类特征进行编码, 不会创建新列
 - 贪心目标编码: 导致目标泄露和预测偏移
 - 保留目标编码 (LOO) : 保留一部分信息进行编码
 - 有序目标编码: 适应类别特征, 对样本进行随机排列, 得到一个训练顺序, 动态计算类别特征的编码值, 仅使用排序中前面的数据
- 目标 (标签) 信息泄露: 当标签信息泄露到测试集
- 预测偏移 (Prediction shift) : 同一样本既用于建树又用于计算梯度, 导致梯度估计不是无偏的
- 有序提升策略: 使用基于排列的方法进行集成, 在不同的数据排列和数据子集上训练一系列模型, 以减少预测偏移, 最终模型通过了多次迭代优化, 减少梯度估计的偏差



3.4 梯度提升算法小结

	GBDT	XGBoost	LightGBM	CatBoost
基本原理	基于梯度提升，串行构建决策树，每棵树拟合负梯度	在GBDT基础上加入二阶泰勒展开和正则化项	基于直方图的梯度提升，采用叶子生长策略	基于对称树和有序提升的梯度提升算法
树模型类型	CART	CART	CART	对称树
分裂算法	精确贪心算法	精确贪心算法，近似算法	直方图算法（256bins）	基于特征排序的贪心算法
类别特征处理	手动	手动	支持类别特征	有序目标编码
防止过拟合	参数调优和正则化	加入L1和L2显式正则化	最大深度限制和特征分箱策略	有序提升
并行运算	支持、不完善	支持、较完善	支持、GPU加速高效	支持GPU
主要优点	理论基础坚实、实现相对简单	精度高、正则化防止过拟合、灵活性强	训练速度快、内存消耗低	类别特征处理强、避免目标泄露、超参数调优简单
局限性	训练速度慢，不支持并行处理	内存占用大，调参复杂	对小数据集不如XGBoost稳定，直方图有精度损失	内存消耗大，对称树灵活性较低
适用场景	小数据集的分类和回归任务	大规模数据集的分类、回归、排序	超大规模数据集，高维任务	包含大量分类特征的数据集

4

集成学习基础



4.1 集成学习方法分类

方法类型

学习器关系

结合方式

目标

• 并行化方法：独立、并行

- 同质并行：Bagging、Random Forest
- 异质并行：Stacking(堆叠)
-

平均/投票
加权/学习法

↓ 方差

• 序列化方法：依赖、串行

- 自适应提升：Adaboost
- 梯度提升：XGBoost、LightGBM、CatBoost
-

加权法

↓ 偏差

个体学习器
“好而不同”

偏差Bias与方差Variance

模型泛化误差分解

记样本量为 n 的数据集为 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, $y = f(x) + \varepsilon$,

构建模型 $\hat{f}$, 希望与真实 f 相近

误差为: $E_D[(y - \hat{f}(x))^2]$

$$= E[(f(x) + \varepsilon - \hat{f}(x))^2]$$

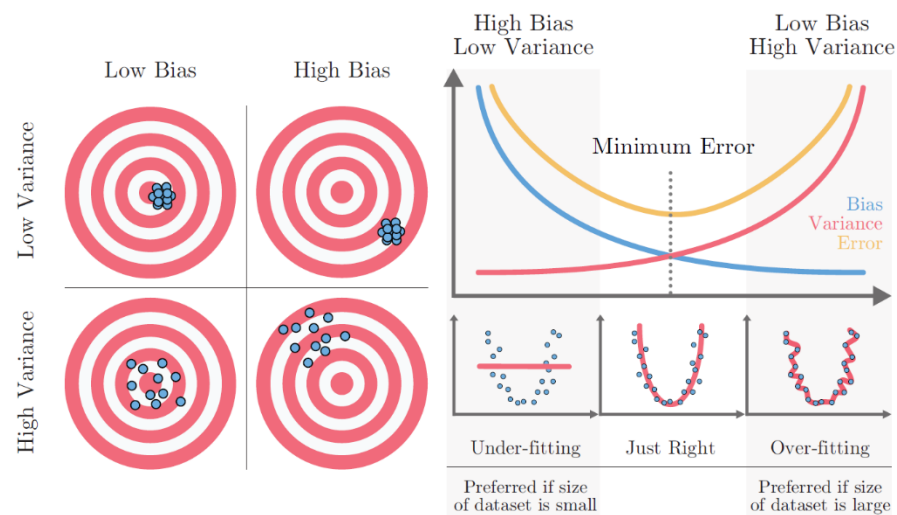
$$= E[((f - E(\hat{f})) + \varepsilon - (\hat{f} - E(\hat{f})))^2]$$

$$= (f - E(\hat{f}))^2 + E(\varepsilon^2) + E[(\hat{f} - E(\hat{f}))^2]$$

$$= \text{Bias}[\hat{f}]^2 + \text{Var}[\hat{f}] + \sigma^2$$

偏差的平方 方差

$$E(f) = f \quad E(\varepsilon) = 0, \text{Var}(\varepsilon) = \sigma^2 \quad \varepsilon \text{ 与 } \hat{f} \text{ 独立}$$



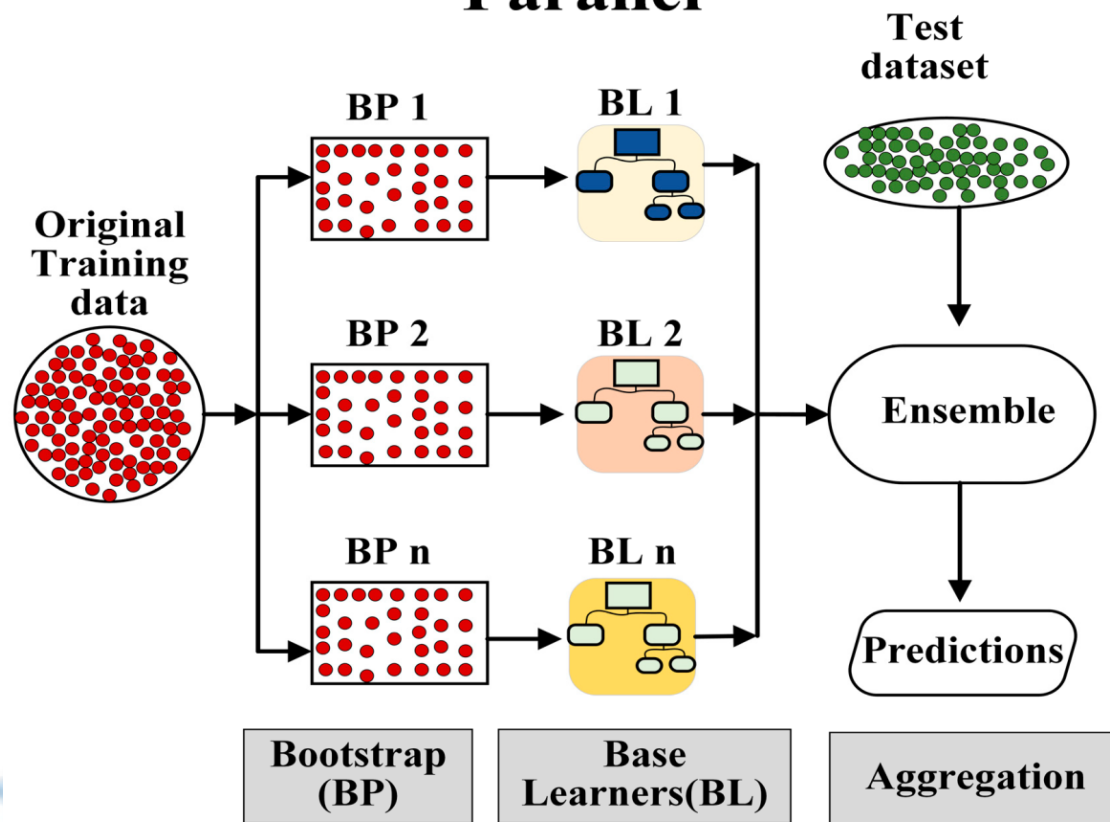
Bias-Variance Tradeoff

- **降低偏差**: 增加模型复杂度、Boosting、Stacking
- **降低方差**: 简单模型、正则化、Bagging、Stacking
- **降低 σ^2** : 提升数据质量

4.1 集成学习方法分类

并行化方法

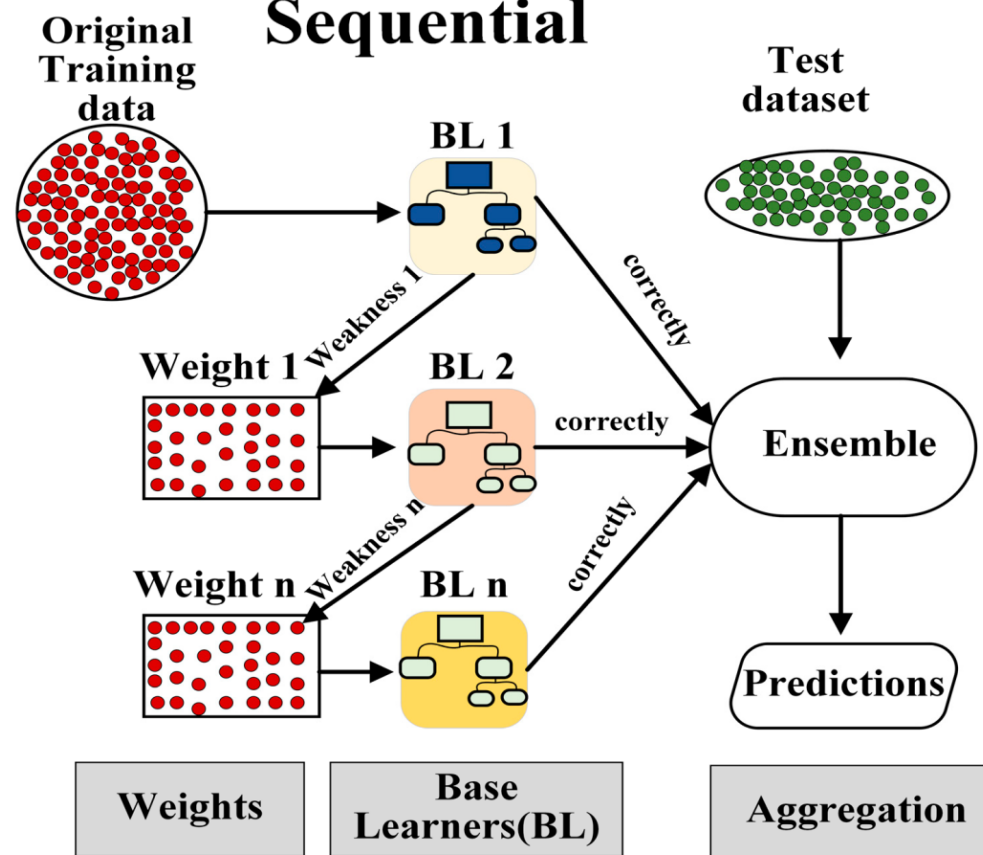
Parallel



同质并行: Bagging、Random Forest
异质并行: Stacking(堆叠)

序列化方法

Sequential



自适应提升: Adaboost
梯度提升: XGBoost、LightGBM、CatBoost



4.2 集成学习在医学中的应用

诊断预测模型

Stroke Machine Learning Prediction of Stroke
Mechanism in Embolic Strokes of Undetermined
Source

机器学习预测未确定来源栓塞性卒中（ESUS）的卒中机制

- **模型：**超级学习器（Super Learner），通过10折交叉验证训练多个基模型（L1 正则化logistic回归、XGBoost、随机森林、多元自适应回归样条MARS、普通logistic回归），加权组合以最小化交叉熵损失
- **训练集**N=1083，**验证集**N=580
- **输入特征：**人口统计学、合并症、生命体征、实验室结果、超声心动图数据
- **预测目标：**心源性栓塞（二分类）
- **模型输出：**每个 ESUS 患者心源性栓塞的预测概率

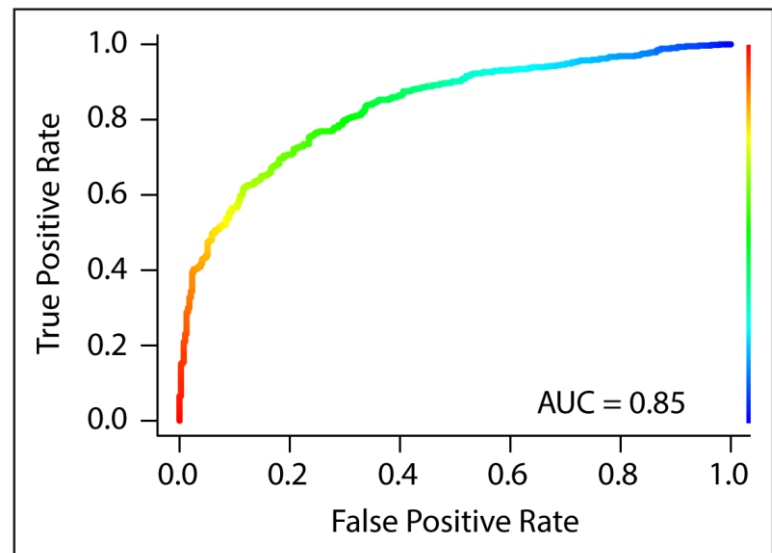


Figure 2. Receiver operating characteristics curve for Bayesian targeted machine learning estimator of probability of occult cardioembolic source. AUC indicates area under the curve.



4.2 集成学习在医学中的应用

诊断预测模型

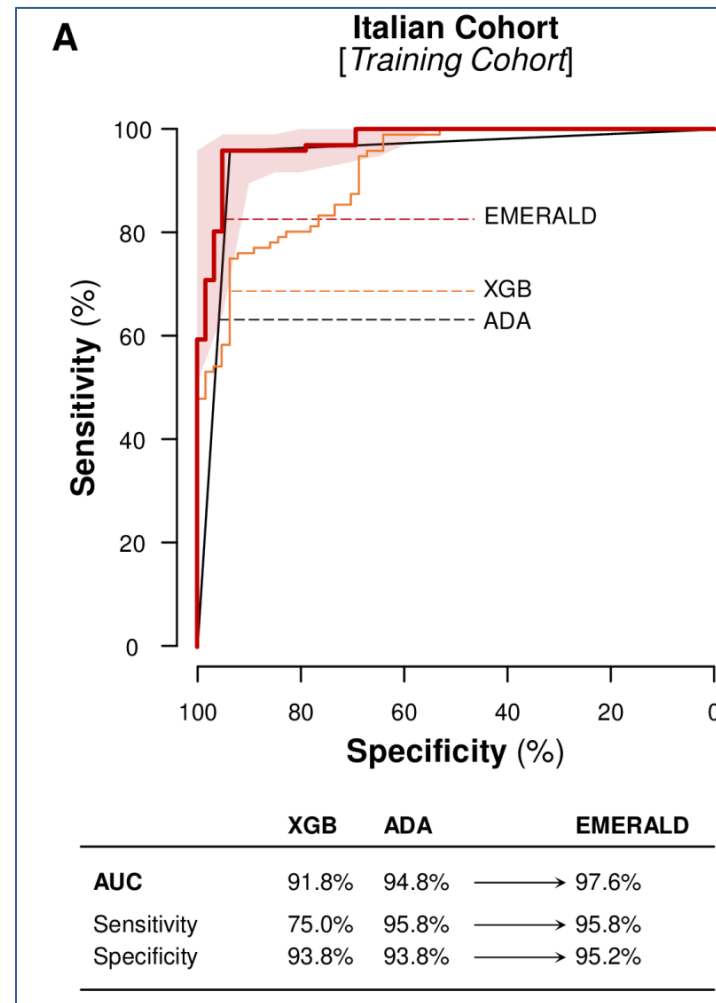
Original research

Gut

Liquid biopsy to identify Barrett's oesophagus, dysplasia and oesophageal adenocarcinoma: the *EMERALD* multicentre study

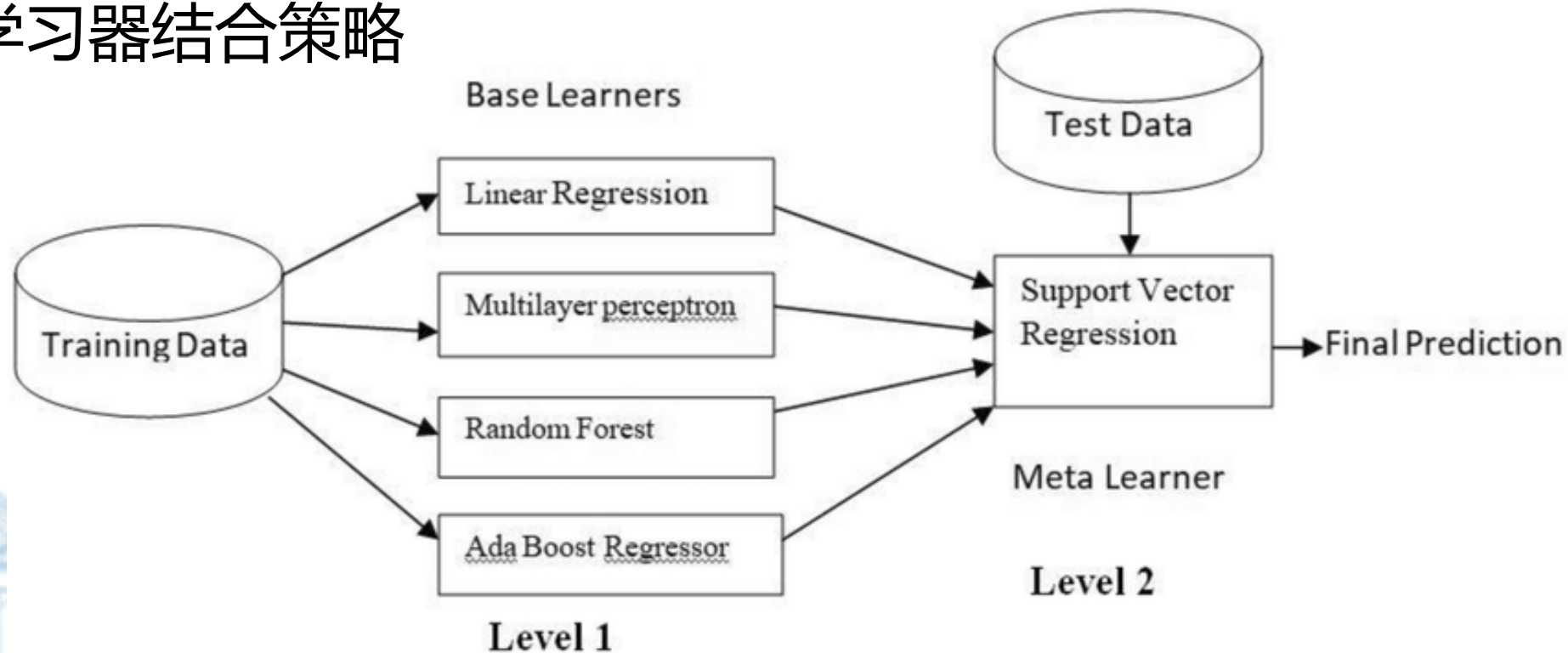
液体活检用于识别巴雷特食管、异型增生和食管腺癌：
EMERALD多中心研究

- **模型**：XGBoost 和 AdaBoost，并采用stacking策略，将两个模型的预测结果通过logistic回归结合成最终模型
- **输入特征**：6种循环miRNA的表达水平
- **预测目标**：食管腺癌/巴雷特食管/其异型增生（二分类）
- **模型输出**：风险分数



4.3 Stacking

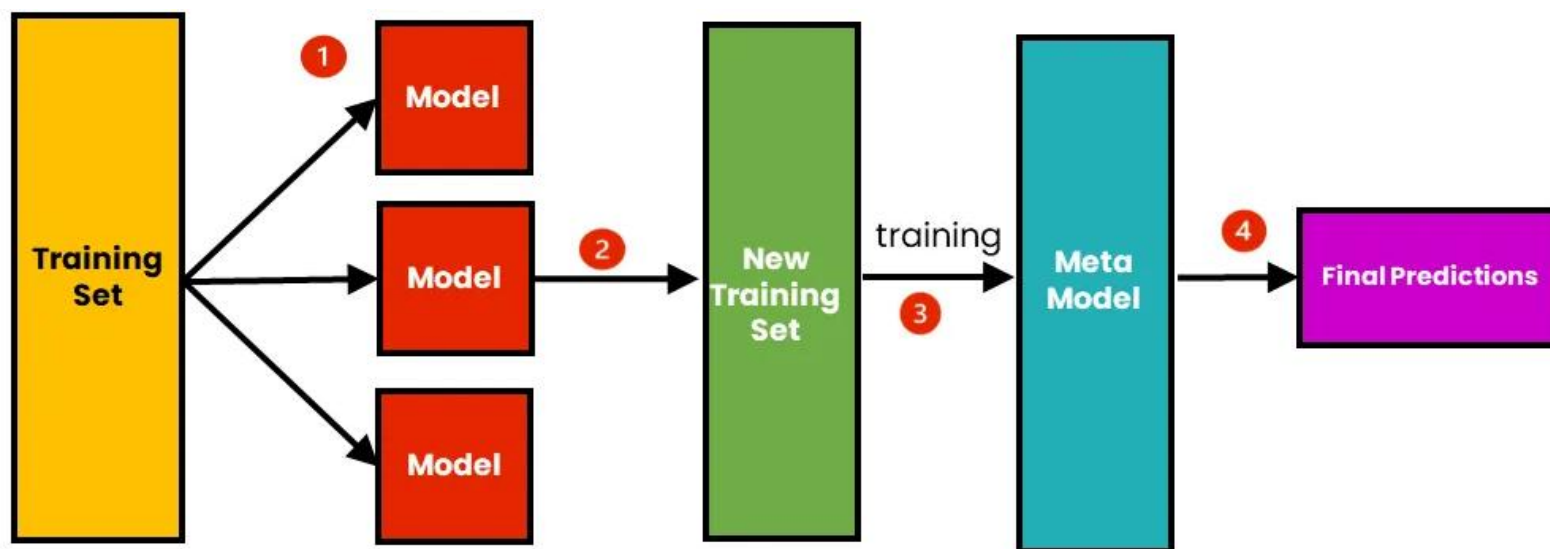
- Stacking（堆叠法）本身是一种著名的集成学习方法，且有不少集成学习算法可视为其变体或特例，它也可看作一种特殊的学习器结合策略



4.3 Stacking

元学习方法：从数据中学习结合函数

- Stacking是最常见的**元学习方法**，一般包含两个步骤：
 - 在训练集数据上拟合基学习器（多样、异质）
 - 根据基学习器的预测结果构建新的数据集，这些预测称为元特征，使用任何基础学习算法进行结合（常用logistic回归）=> 通过**交叉验证**进行stacking避免过拟合（如Super Learner采用V折交叉验证）=> Blending



5

案例分析

5.1 威斯康星乳腺癌数据集

	ID	Diagnosis	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	s
1	842302	M	17.990	10.38	122.80	1001.0	0.11840	0.27760	0.300100	0.147100	
2	842517	M	20.570	17.77	132.90	1326.0	0.08474	0.07864	0.086900	0.070170	
3	84300903	M	19.690	21.25	130.00	1203.0	0.10960	0.15990	0.197400	0.127900	
4	84348301	M	11.420	20.38	77.58	386.1	0.14250	0.28390	0.241400	0.105200	
5	84358402	M	20.290	14.34	135.10	1297.0	0.10030	0.13280	0.198000	0.104300	
6	843786	M	12.450	15.70	82.57	477.1	0.12780	0.17000	0.157800	0.080890	
7	844359	M	18.250	19.98	119.60	1040.0	0.09463	0.10900	0.112700	0.074000	
8	84458202	M	13.710	20.83	90.20	577.9	0.11890	0.16450	0.093660	0.059850	
9	844981	M	13.000	21.82	87.50	519.8	0.12730	0.19320	0.185900	0.093530	
10	84501001	M	12.460	24.04	83.97	475.9	0.11860	0.23960	0.227300	0.085430	
11	845636	M	16.020	23.24	102.70	797.8	0.08206	0.06669	0.032990	0.033230	
12	84610002	M	15.780	17.89	103.60	781.0	0.09710	0.12920	0.099540	0.066060	
13	846226	M	19.170	24.80	132.40	1123.0	0.09740	0.24580	0.206500	0.111800	
14	846381	M	15.850	23.95	103.70	782.7	0.08401	0.10020	0.099380	0.053640	
15	84667401	M	13.730	22.61	93.60	578.3	0.11310	0.22930	0.212800	0.080250	
16	84799002	M	14.540	27.54	96.73	658.8	0.11390	0.15950	0.163900	0.073640	
17	848406	M	14.680	20.13	94.74	684.5	0.09867	0.07200	0.073950	0.052590	
18	84862001	M	16.130	20.68	108.10	798.8	0.11700	0.20220	0.172200	0.102800	
19	849014	M	19.810	22.15	130.00	1260.0	0.09831	0.10270	0.147900	0.094980	
20	8510426	B	13.540	14.36	87.46	566.3	0.09779	0.08129	0.066640	0.047810	
21	8510653	B	13.080	15.71	85.63	520.0	0.10750	0.12700	0.045680	0.031100	
22	8510824	B	9.504	12.44	60.34	273.9	0.10240	0.06492	0.029560	0.020760	
23	8511133	M	15.340	14.26	102.50	704.4	0.10730	0.21350	0.207700	0.097560	

```
[1] "ID" "Diagnosis"
[3] "radius1" "texture1"
[5] "perimeter1" "area1"
[7] "smoothness1" "compactness1"
[9] "concavity1" "concave_points1"
[11] "symmetry1" "fractal_dimension1"
[13] "radius2" "texture2"
[15] "perimeter2" "area2"
[17] "smoothness2" "compactness2"
[19] "concavity2" "concave_points2"
[21] "symmetry2" "fractal_dimension2"
[23] "radius3" "texture3"
[25] "perimeter3" "area3"
[27] "smoothness3" "compactness3"
[29] "concavity3" "concave_points3"
[31] "symmetry3" "fractal_dimension3"
```

标签: Diagnosis

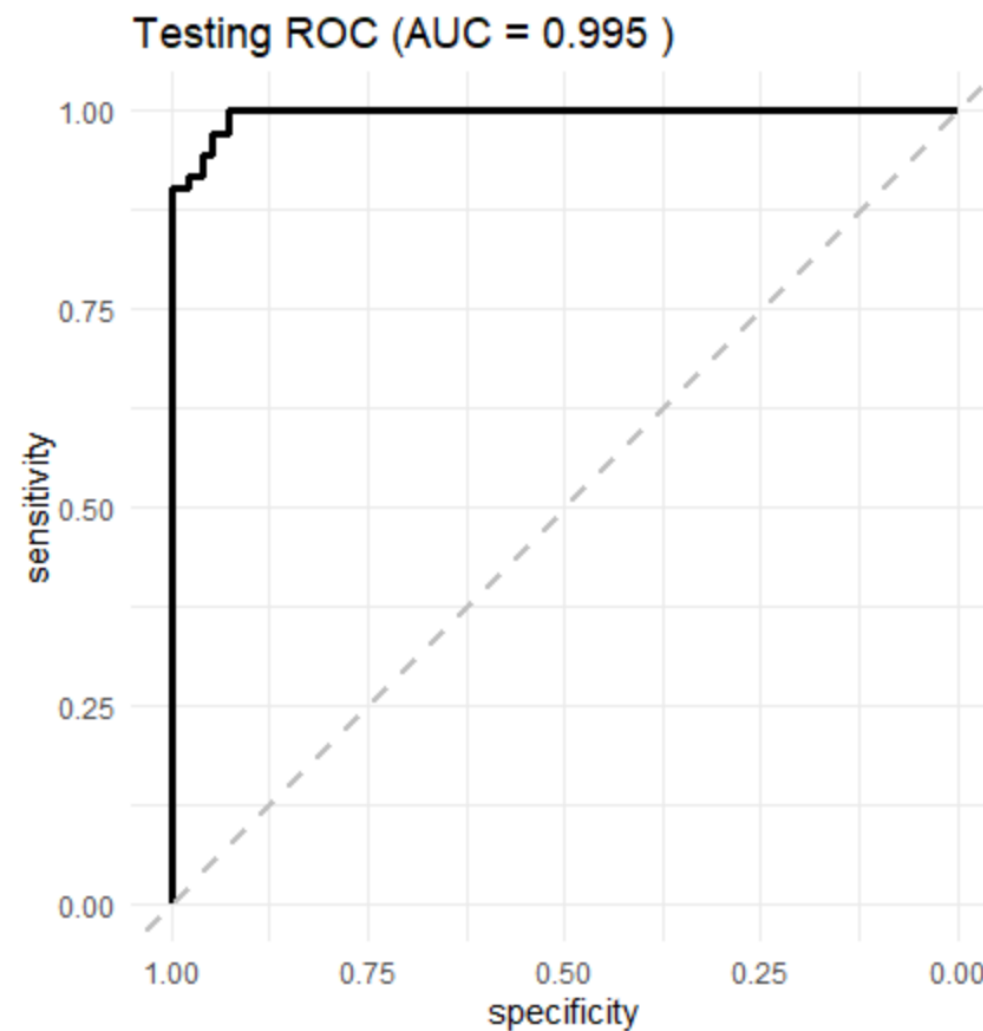
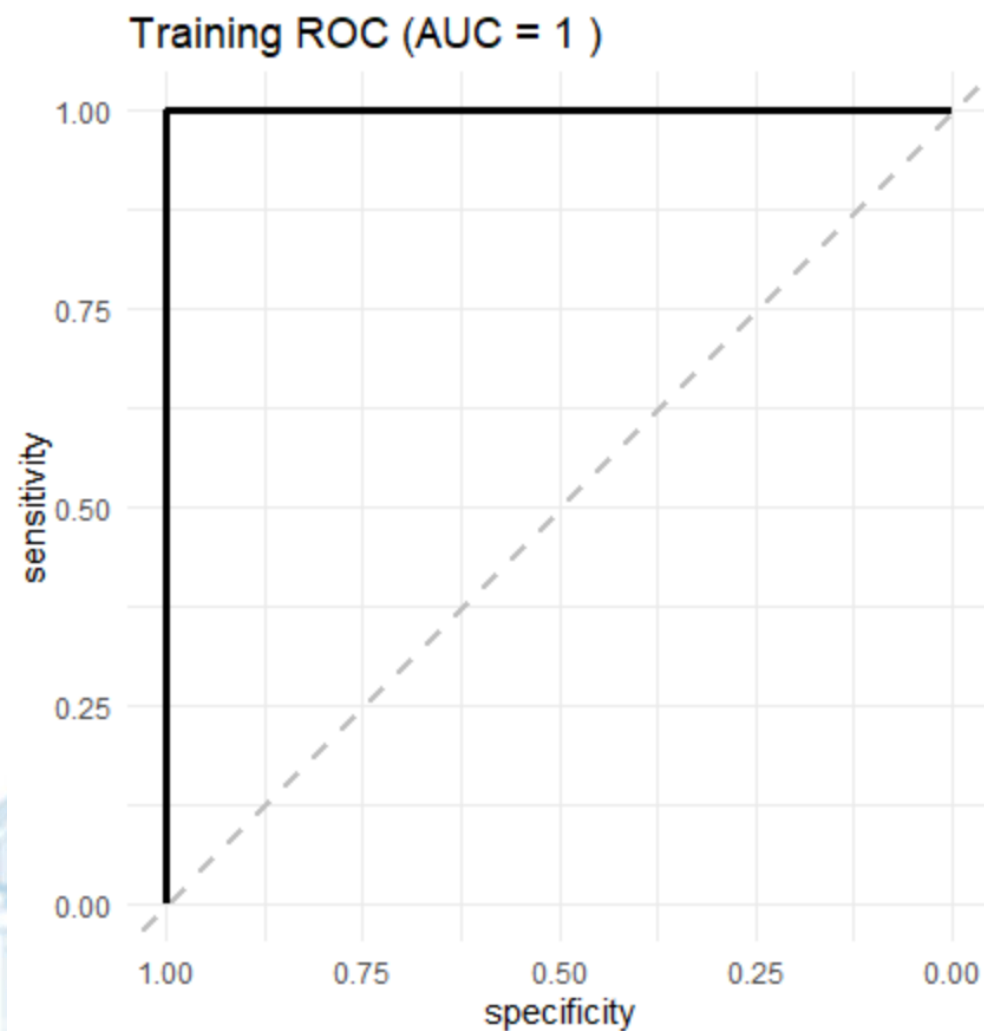
特征: 30个

N=569 (无缺失值)



5.1 威斯康星乳腺癌数据集

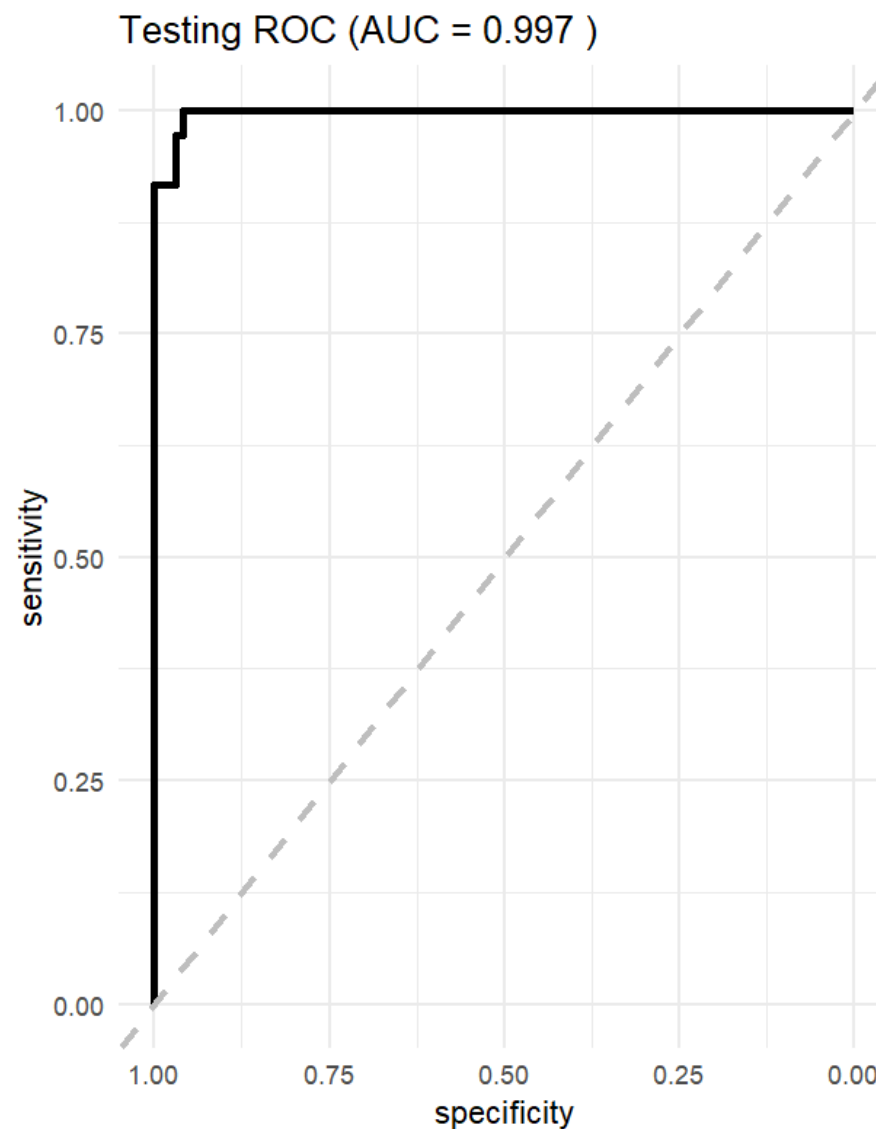
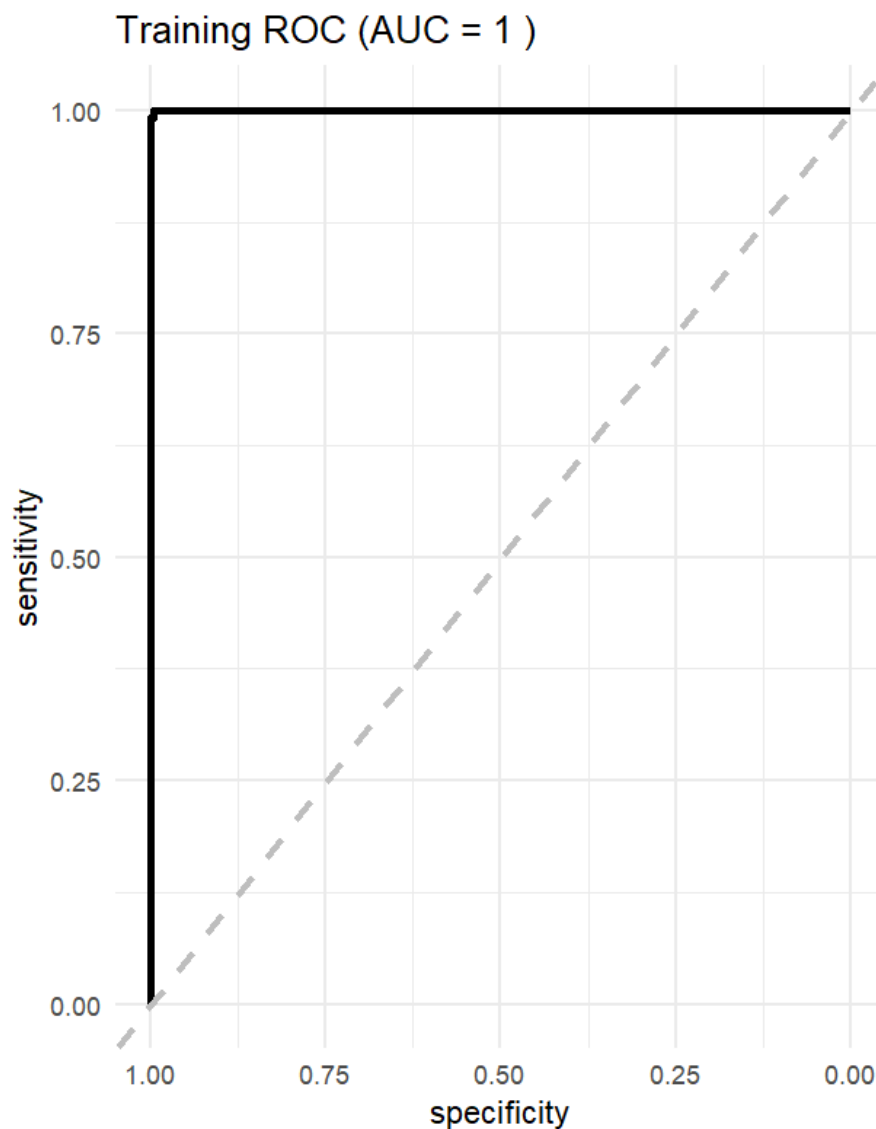
GBDT





5.1 威斯康星乳腺癌数据集

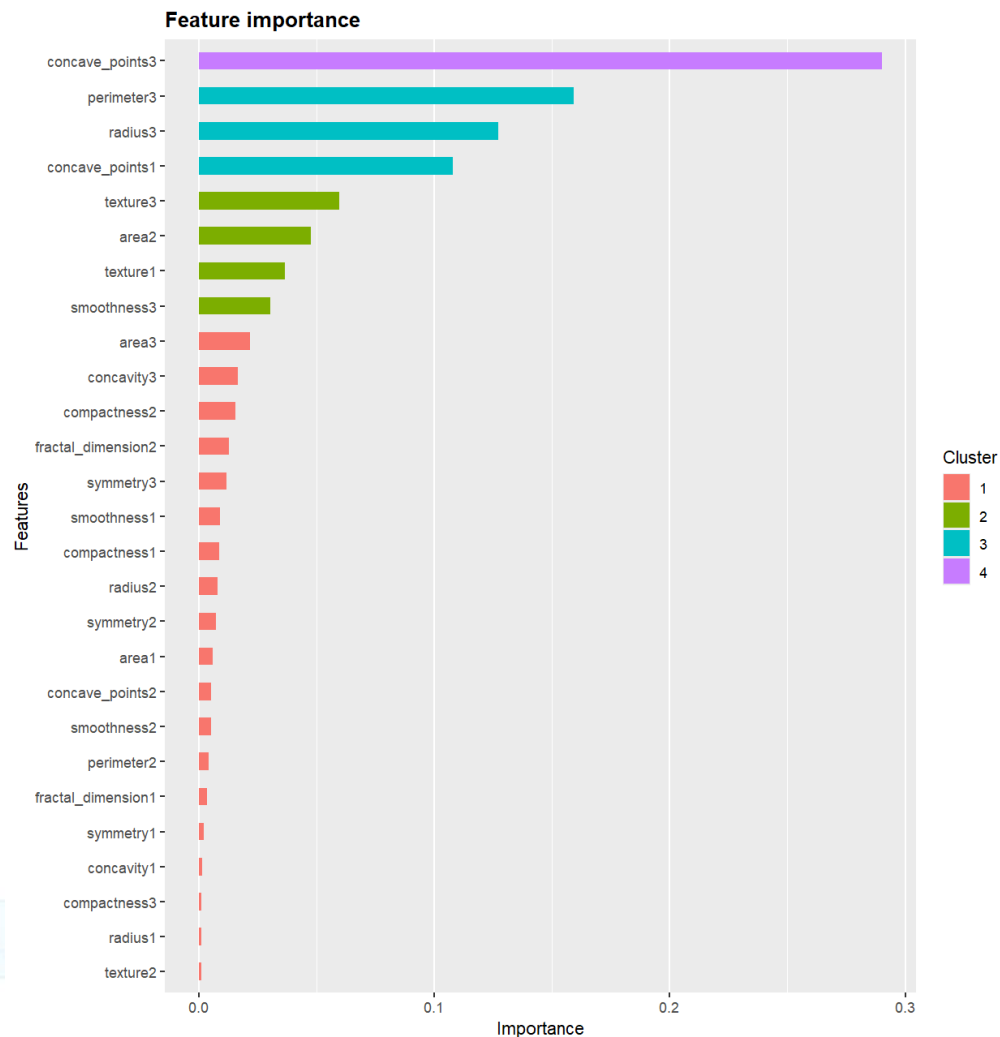
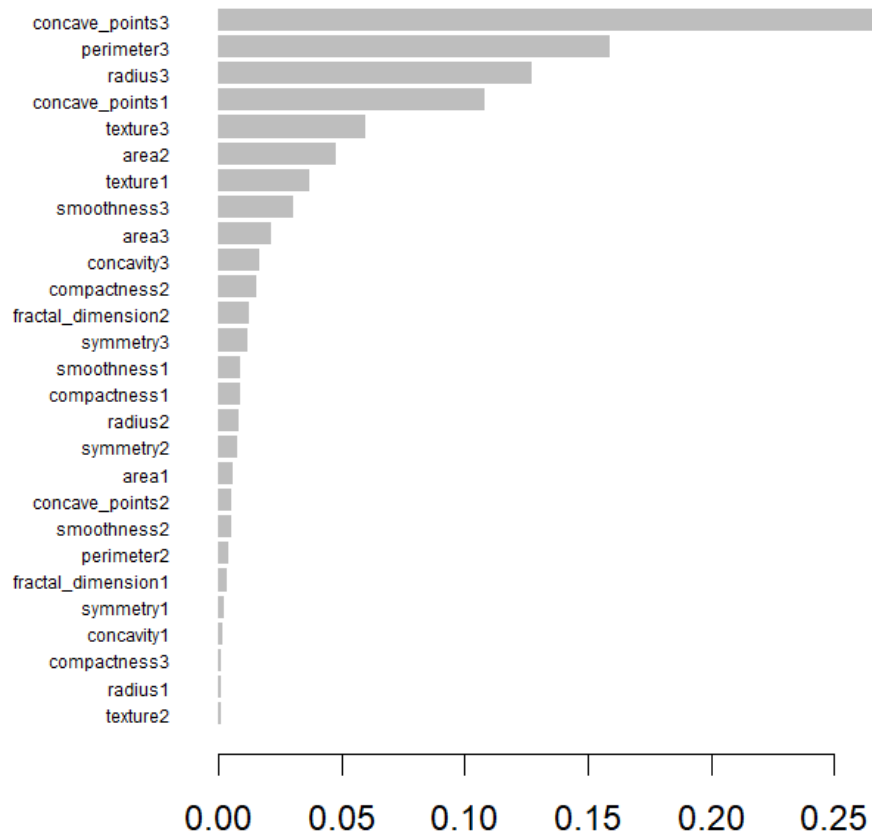
XGBoost-调参后





5.1 威斯康星乳腺癌数据集

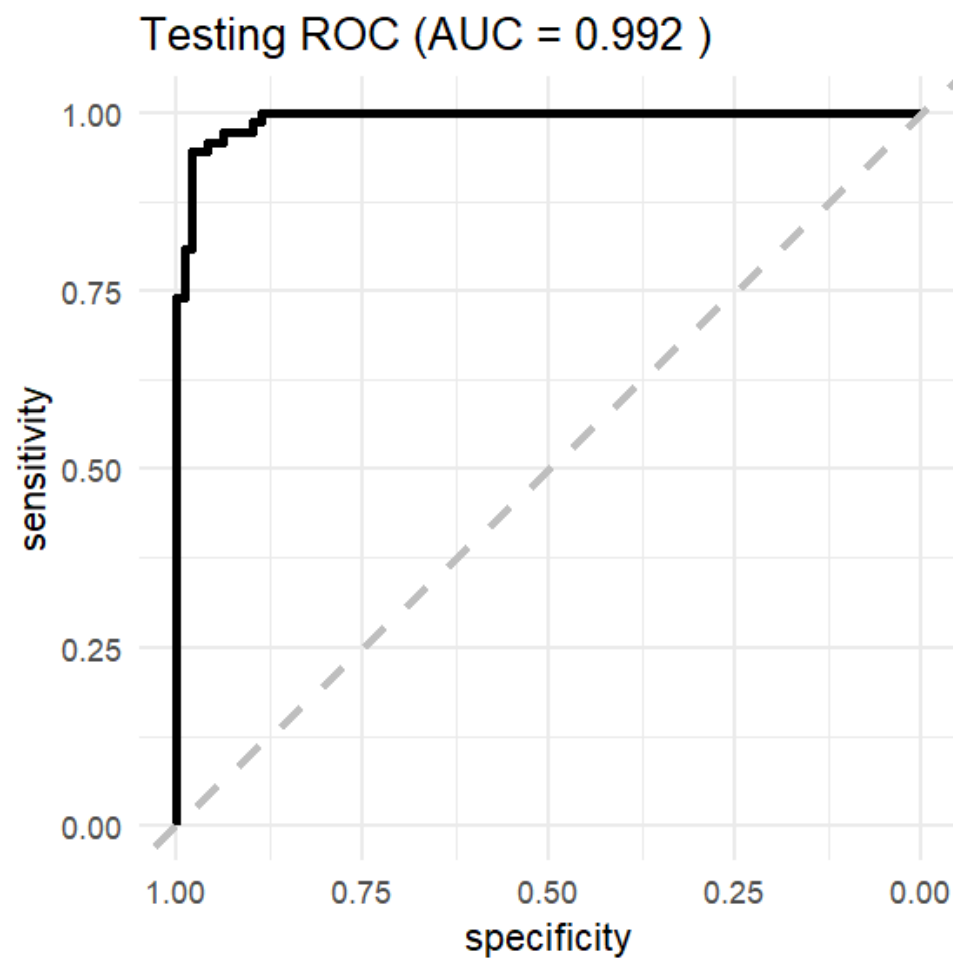
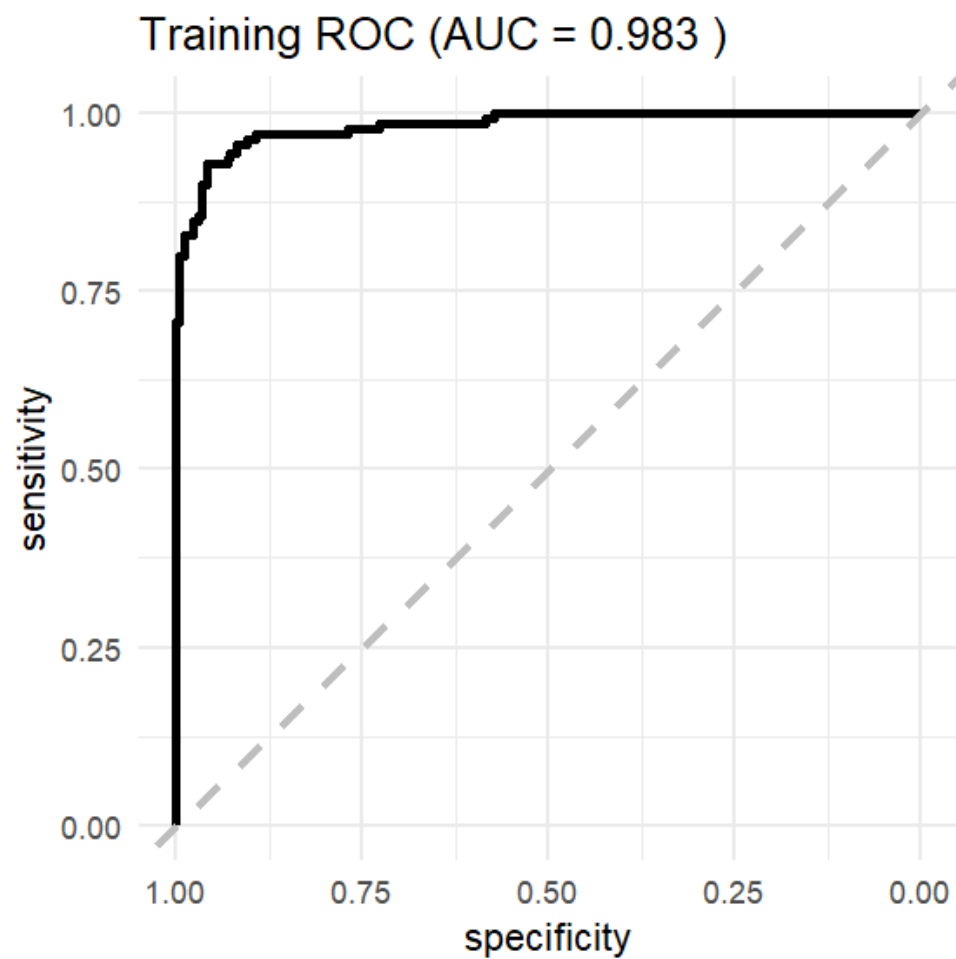
XGBoost-调参后 变量重要性排序





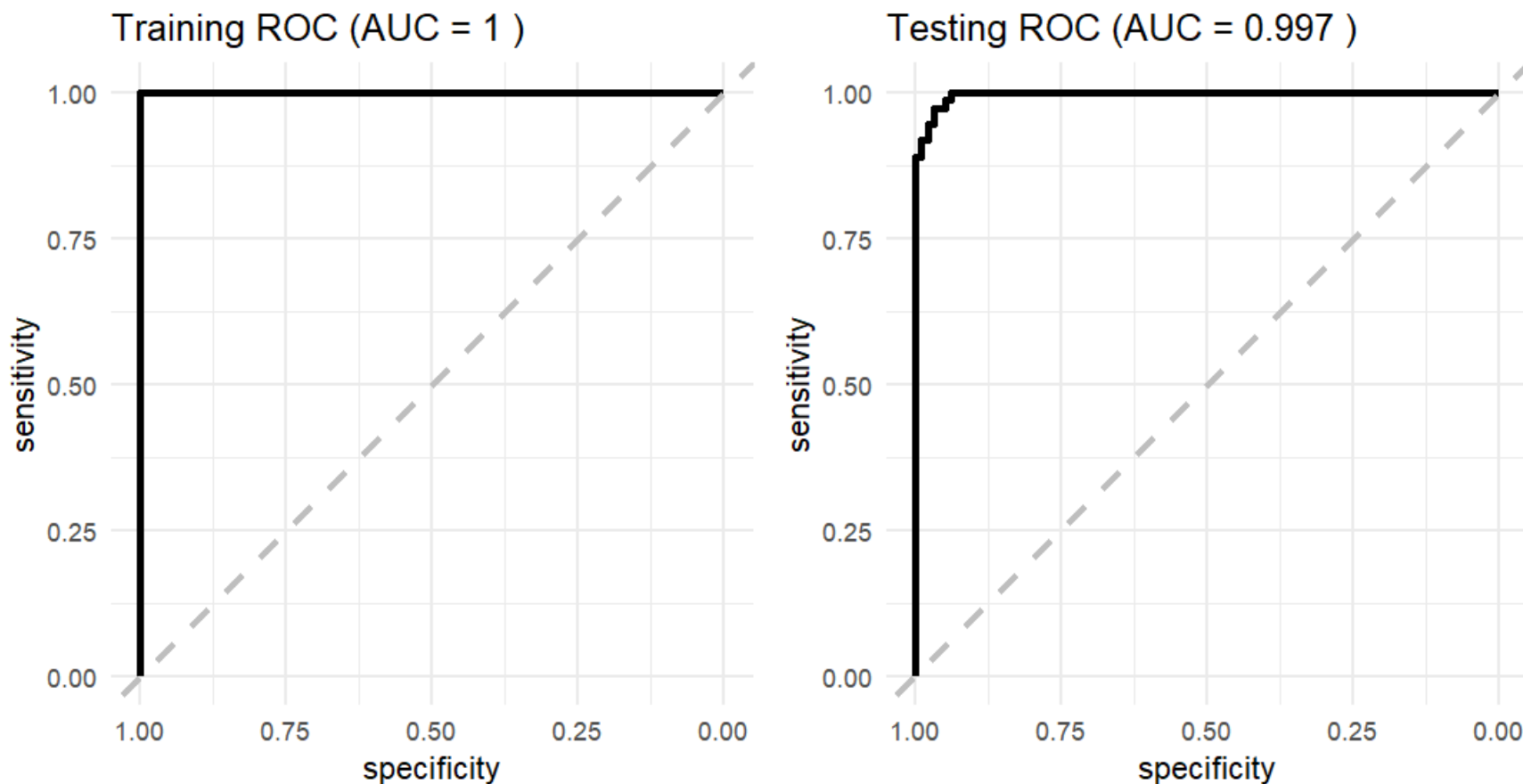
5.1 威斯康星乳腺癌数据集

LightGBM



5.1 威斯康星乳腺癌数据集

CatBoost



XGBoost、LightGBM、CatBoost均获得比GBDT较高的AUC，需进一步进行检验是否有统计学意义

5.2 糖尿病数据集

	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
4	1	89	66	23	94	28.1	0.167	21	neg
5	0	137	40	35	168	43.1	2.288	33	pos
7	3	78	50	32	88	31.0	0.248	26	pos
9	2	197	70	45	543	30.5	0.158	53	pos
14	1	189	60	23	846	30.1	0.398	59	pos
15	5	166	72	19	175	25.8	0.587	51	pos
17	0	118	84	47	230	45.8	0.551	31	pos
19	1	103	30	38	83	43.3	0.183	33	neg
20	1	115	70	30	96	34.6	0.529	32	pos
21	3	126	88	41	235	39.3	0.704	27	neg
25	11	143	94	33	146	36.6	0.254	51	pos
26	10	125	70	26	115	31.1	0.205	41	pos
28	1	97	66	15	140	23.2	0.487	22	neg
29	13	145	82	19	110	22.2	0.245	57	neg
32	3	158	76	36	245	31.6	0.851	28	pos
33	3	88	58	11	54	24.8	0.267	22	neg
36	4	103	60	33	192	24.0	0.966	33	neg
40	4	111	72	47	207	37.1	1.390	56	pos
41	3	180	64	25	70	34.0	0.271	26	neg
44	9	171	110	24	240	45.4	0.721	54	pos
51	1	103	80	11	82	19.4	0.491	22	neg

标签: diabetes

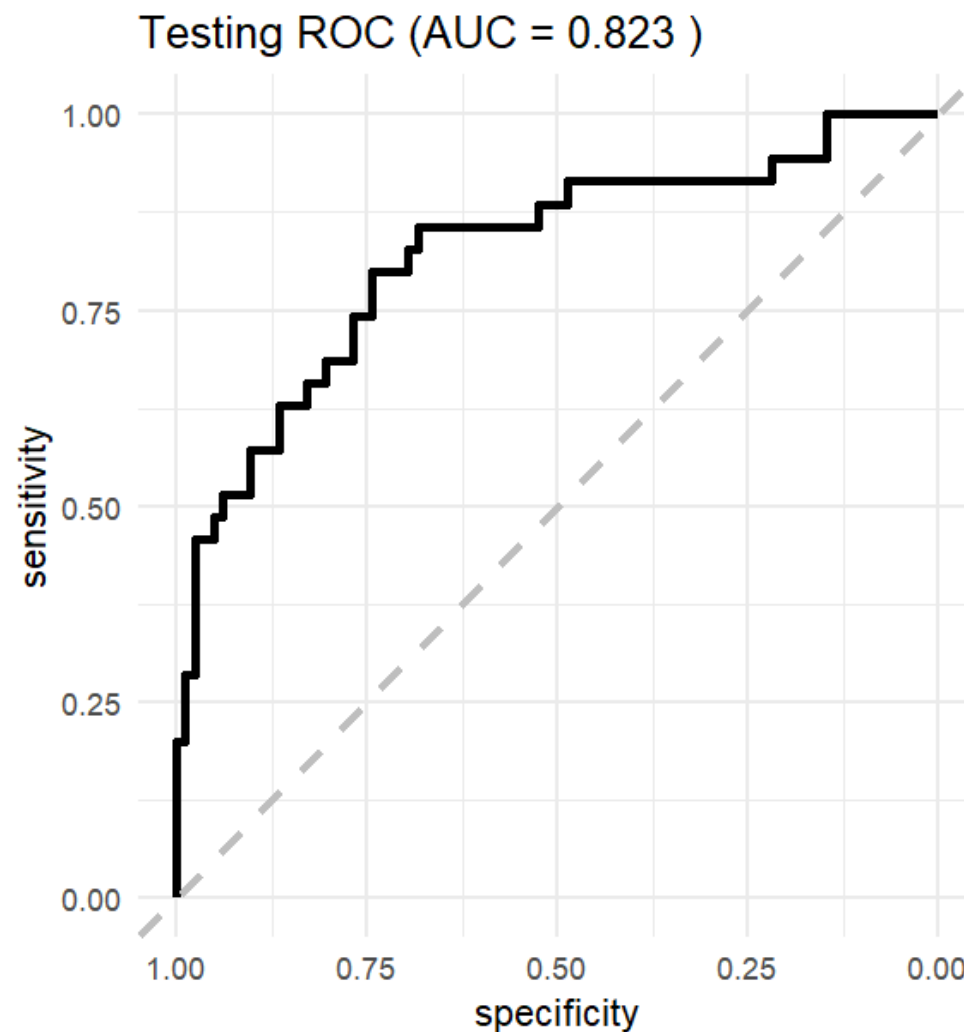
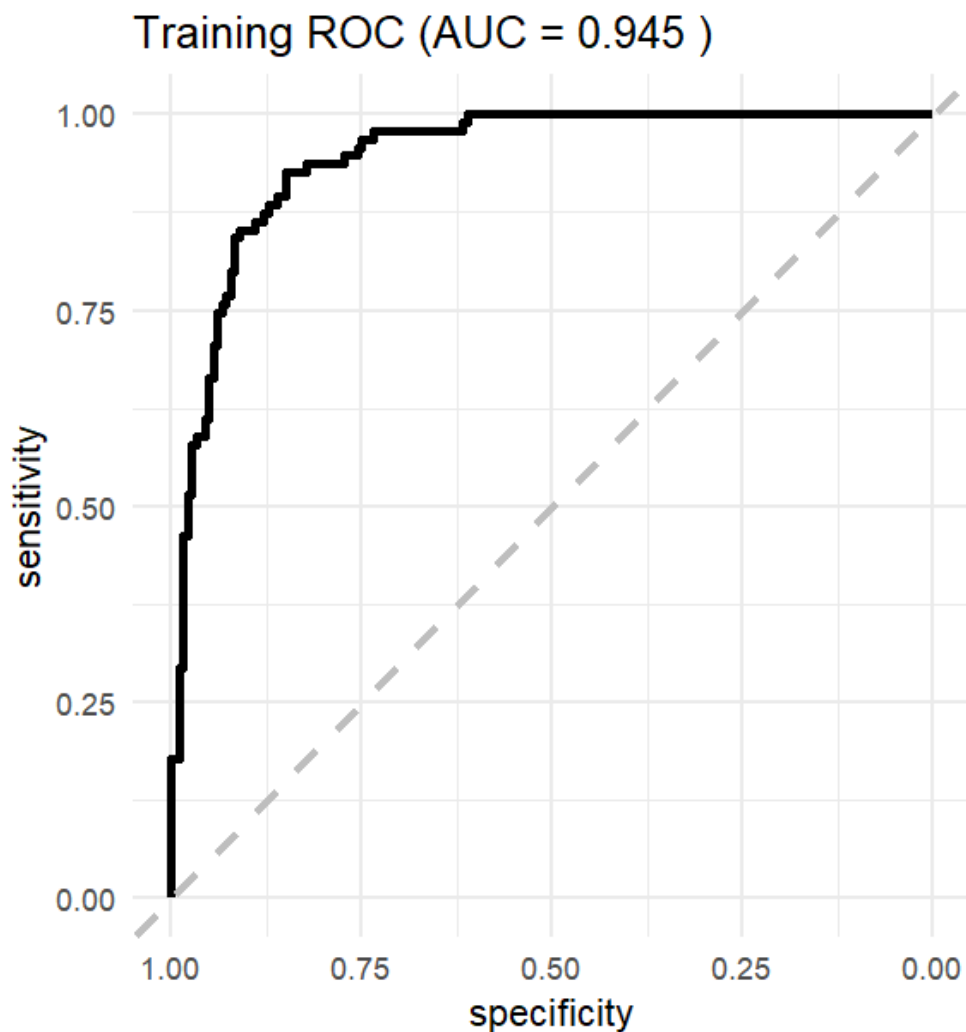
特征: 8个

N=392 (去掉缺失值后)



5.2 糖尿病数据集

GBDT



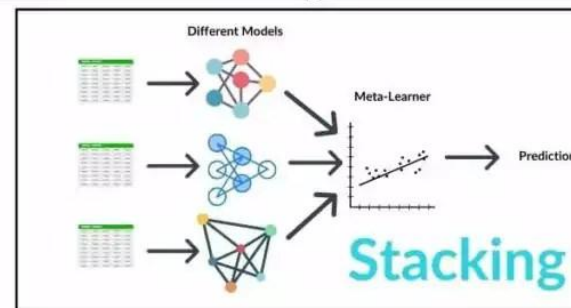
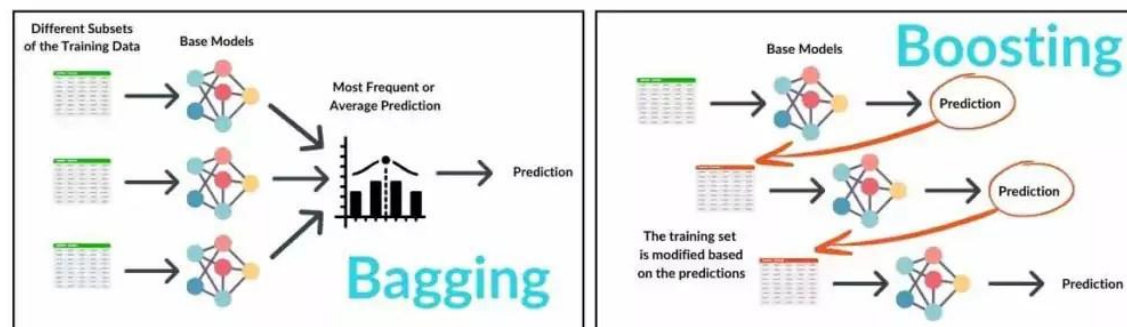
仅截取了GBDT的结果，其他模型结果需要运行代码并调参

• 梯度提升算法：

- GBDT：通过迭代地训练决策树，并将前一棵树的残差作为下一棵树的训练目标，逐步减少预测误差。沿着损失函数的负梯度方向，逐步构建新的决策树来修正前面模型的不足，将训练好的弱分类器以累加的形式结合到现有模型中
- 高效实现：XGBoost、LightGBM、CatBoost

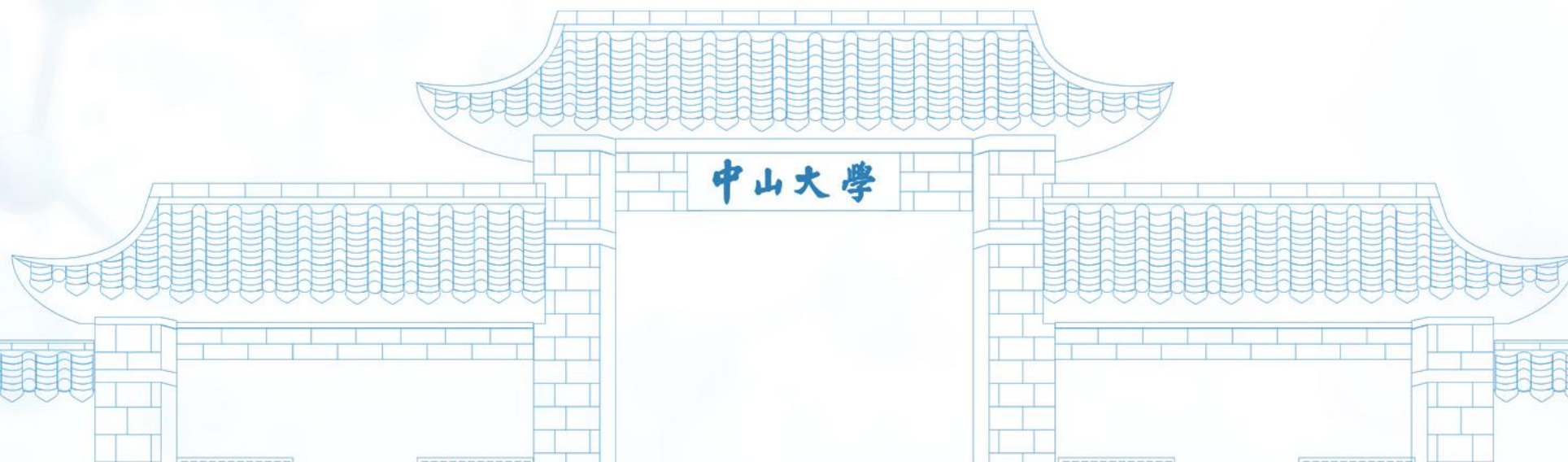
• 集成学习基础：

- Bagging vs Boosting
- 并行化方法 vs 序列化方法
- 数据融合策略：平均、加权以及Stacking





谢谢！





问题与讨论

